

분산처리

Parallel Programming Basics



Computer & Ai

Department of Computer Engineering

주제: 병렬 프로그램 최적화 작성에 대한 사례 연구

병렬 프로그램 작성하기:

병렬 프로그래밍이란 어떤 문제를 가지고 동시에 여러 가지 작업을 수행하여 문제를 해결할 수 있는 프로그램을 만드는 것을 의미합니다. 이를 “병렬 처리”라고 합니다.

최적화:

이것이 바로 중요한 부분입니다. 단순히 병렬 프로그램을 만드는 것이 아니라 빠르고 효율적으로 만드는 것이 중요합니다. 여기에는 낭비되는 시간과 리소스를 줄이기 위한 많은 영리한 기술이 포함됩니다.

▪ 두 가지 프로그래밍 모델

- 데이터 병렬

- 공유 주소 공간

데이터 병렬: 데이터 병렬은 여러 데이터에 대해 동시에 동일한 연산을 수행하는 방식입니다.
→예) 이미지의 모든 픽셀에 동시에 동일한 필터를 적용하는 것과 같다고 생각하면 됩니다.

공유 주소 공간: 프로그램의 여러 부분이 동일한 메모리에 액세스할 수 있는 모델입니다.
→예) 모든 사람이 정보를 읽고 쓸 수 있는 공유 화이트보드가 있는 것과 같습니다.

Creating a parallel program

▪ 병렬화 사고(고려) 과정:

1. 병렬로 수행할 수 있는 작업 파악하기
2. 작업(및 작업과 관련된 데이터)을 분할.
3. 데이터 액세스, 통신 및 동기화 관리



병렬 프로그램을 최적화하려면 작업의 병렬성, 효율성, 비용, 전력 소비 등을 고려해야 하며, 특히 **병렬 알고리즘 설계**가 핵심 역할

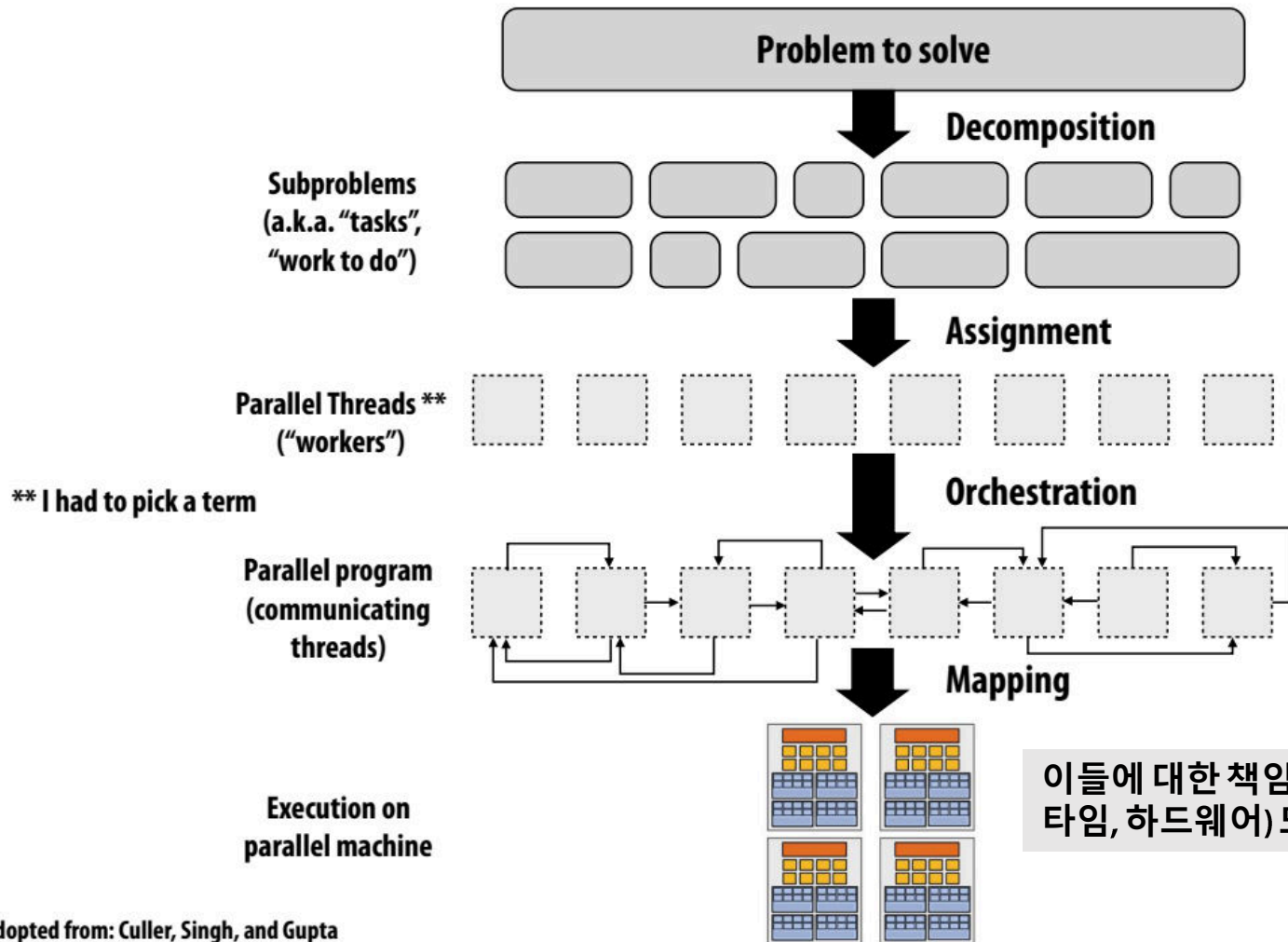
▪ 공통된 목표는 속도 향상 극대화*

For a fixed computation:

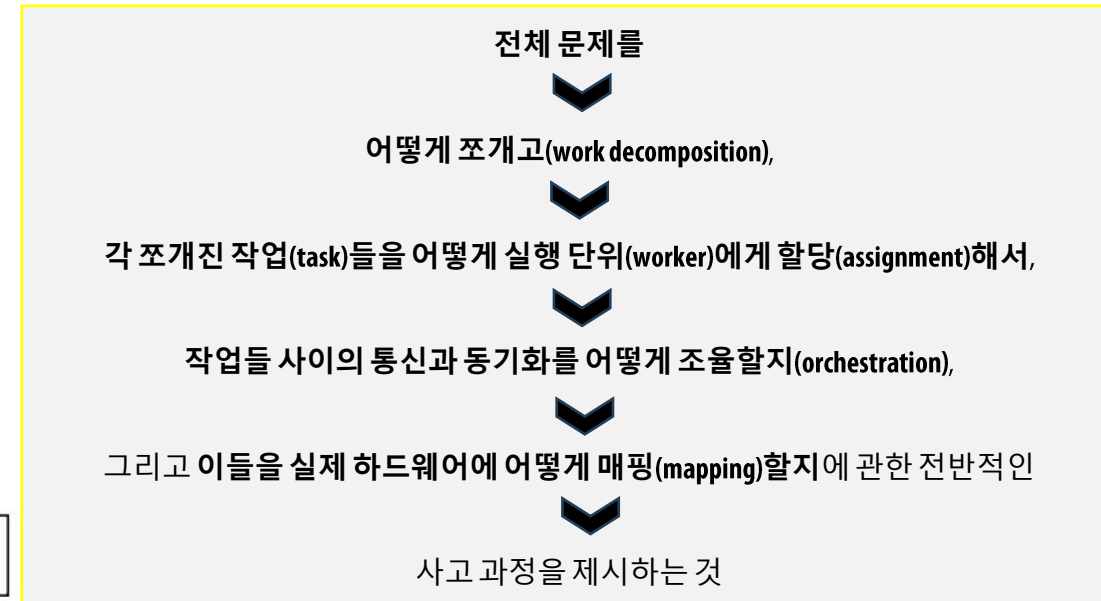
$$\text{Speedup}(P \text{ processors}) = \frac{\text{Time (1 processor)}}{\text{Time (P processors)}}$$

* 다른 목표로는 높은 효율성(비용, 면적, 전력 등)을 달성하거나 한 대의 컴퓨터로 해결할 수 없는 더 큰 문제를 해결하는 것 등이 있음

Creating a parallel program

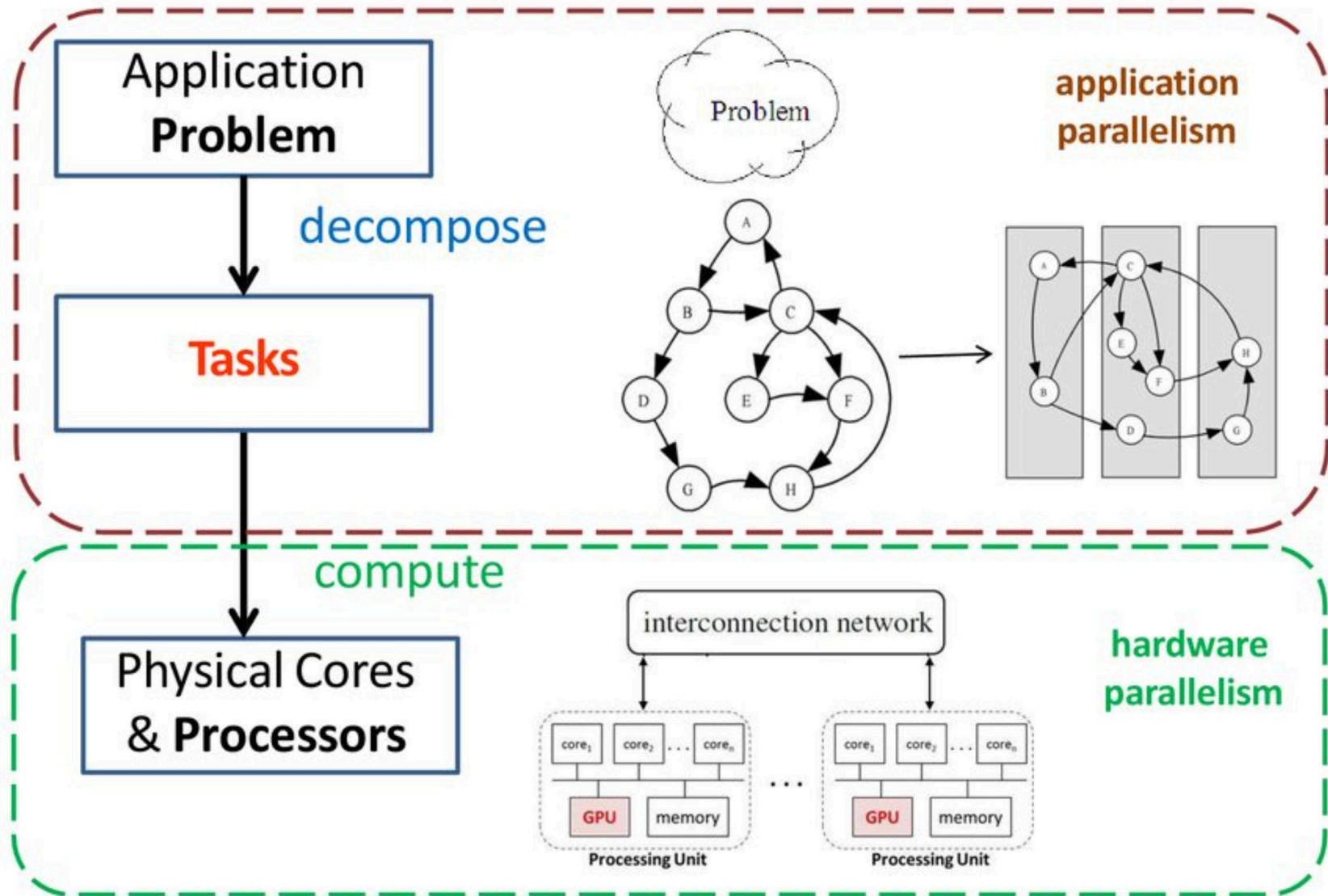


병렬 프로그램을 작성할 때



이들에 대한 책임은 프로그래머, 시스템(컴파일러, 런타임, 하드웨어) 또는 둘 다에 의해 수행될 수 있음

Creating a parallel program



Problem decomposition

- 문제를 병렬로 수행할 수 있는 작업으로 나누기
- 일반적으로: 머신의 모든 실행 유닛을 바쁘게 유지(실행)할 수 있을 만큼 충분한 작업을 생성.

분해(decomposition)의 핵심 과제:

종속성(의존성) 식별

- 분할의 주요 과제는 의존성을 식별하는 것.
- 의존성이 없거나 적은 작업을 찾아내는 것이 중요.

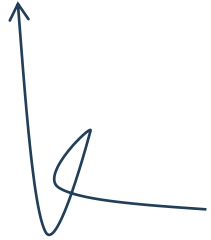
Amdahl's Law: 종속성(의존성) 이 병렬성에 의한 최대 속도 향상 제한

- 문제를 병렬로 수행할 수 있는 작업으로 나누기
- 일반적으로: 머신의 모든 실행 유닛을 바쁘게 유지할 수 있을 만큼 충분한 작업을 생성.

▪ 선호하는 순차 프로그램을 실행...

▪ **S** = 고유한 순차적인 실행의 비율 (종속성으로 인해 병렬 실행이 제한됨)이라고 가정.

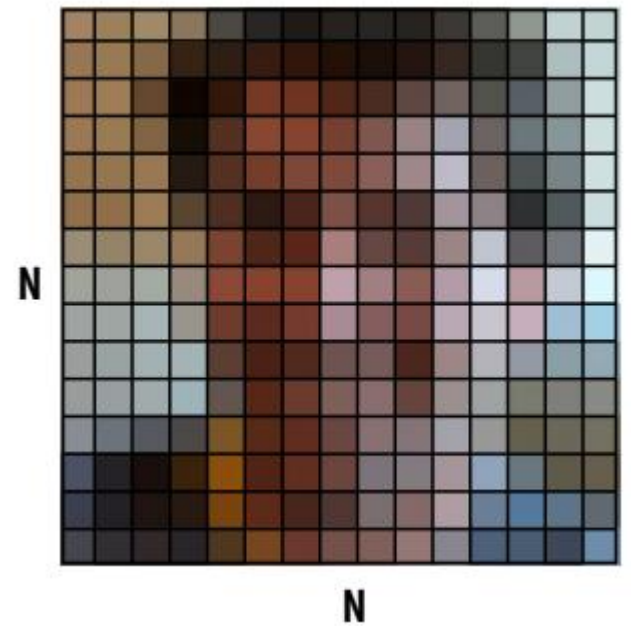
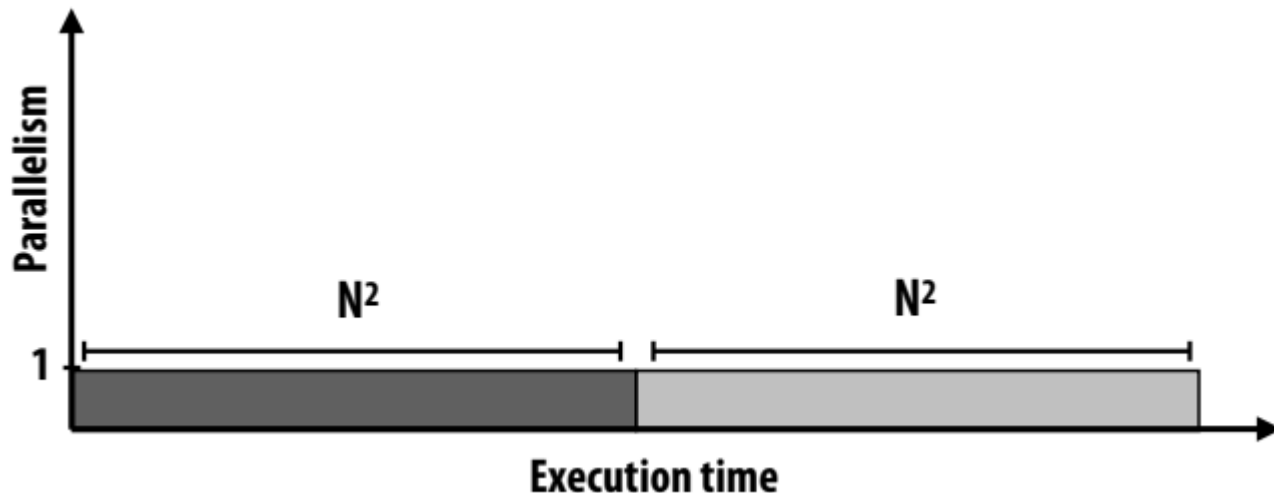
▪ 따라서 병렬 실행으로 인한 최대 속도 향상 $\leq 1/s$



프로그램의 일부가 순차적으로 실행되어야 한다면, 병렬 실행의 최대 속도 향상은 그 부분에 의해 제한됨

간단한 예

- $N \times N$ 이미지에 대한 2단계 계산을 예를 살펴보자.
 - 1단계: 모든 픽셀의 밝기에 2를 곱함.
(각 픽셀에 대해 독립적으로 계산)
 - 2단계: 모든 픽셀 값의 평균 계산
- 프로그램의 순차적 구현
 - 두 단계 모두 $\sim N^2$ 시간이 소요되므로 총 시간은 $\sim 2N^2$



병렬 처리의 첫 시도 (P processors)

▪ Strategy:

- 1단계: 병렬 실행

➤ 1단계 시간: $\frac{N^2}{P}$

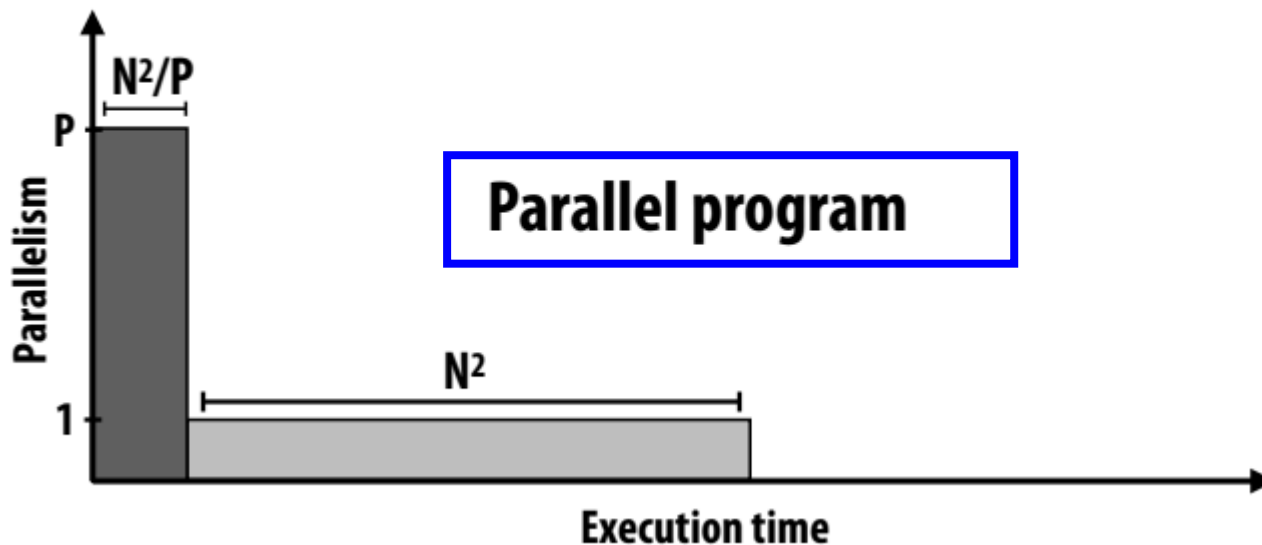
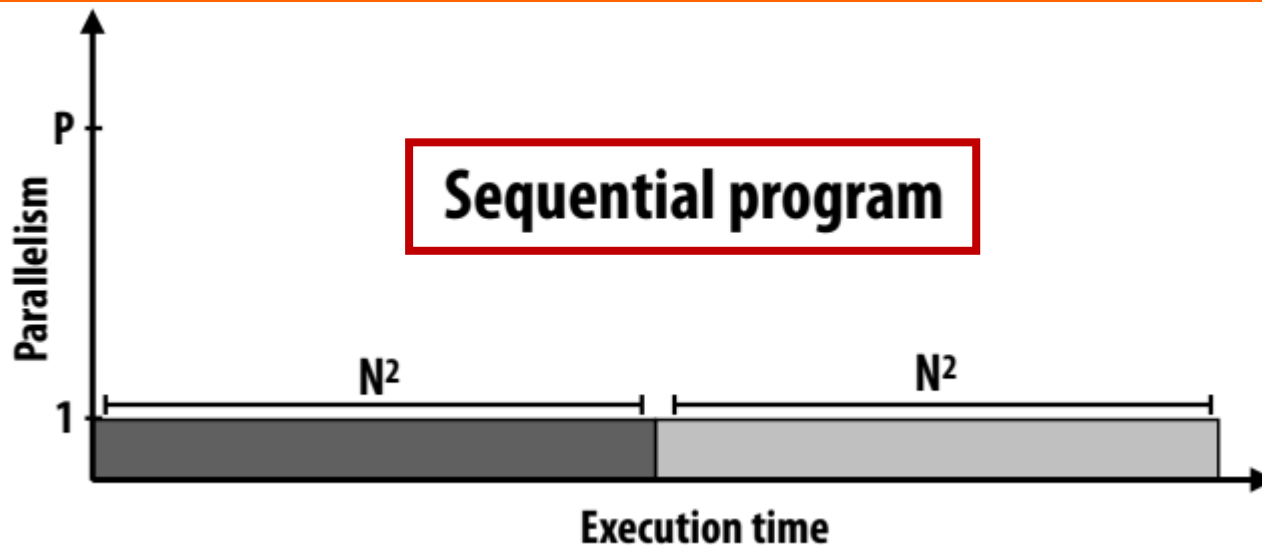
- 2단계: 순차 실행

➤ 2단계 시간: N^2

▪ 전반적인 성능:

$$\text{Speedup} \leq \frac{2n^2}{\frac{n^2}{p} + n^2}$$

$$\text{Speedup} \leq 2$$



Parallelizing step 2

- Strategy:

- 1단계: 병렬 실행

- 1단계 시간: $\frac{N^2}{P}$

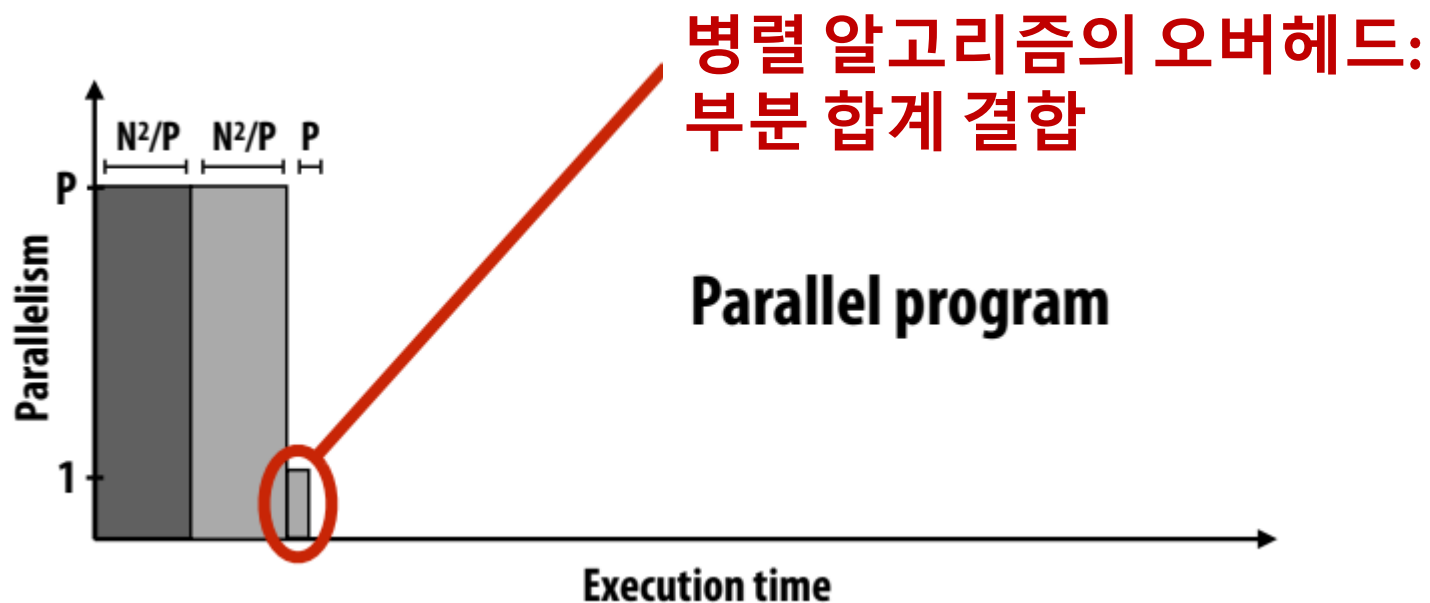
- 2단계: 부분 합계를 병렬로 계산하고, 그 결과를 연속적으로 결합

- 2단계 시간: $\frac{N^2}{P} + P$

- 전반적인 성능:

- $$\text{Speedup} \leq \frac{2n^2}{\frac{2n^2}{p} + p}$$

Note: speedup $\rightarrow P$ when $N \gg P$



Amdahl's law

▪ s = 순차 계산작업의 비율

▪ p 프로세서의 최대 속도 향상:

$$\text{speedup} \leq \frac{1}{s + \frac{1-s}{p}}$$

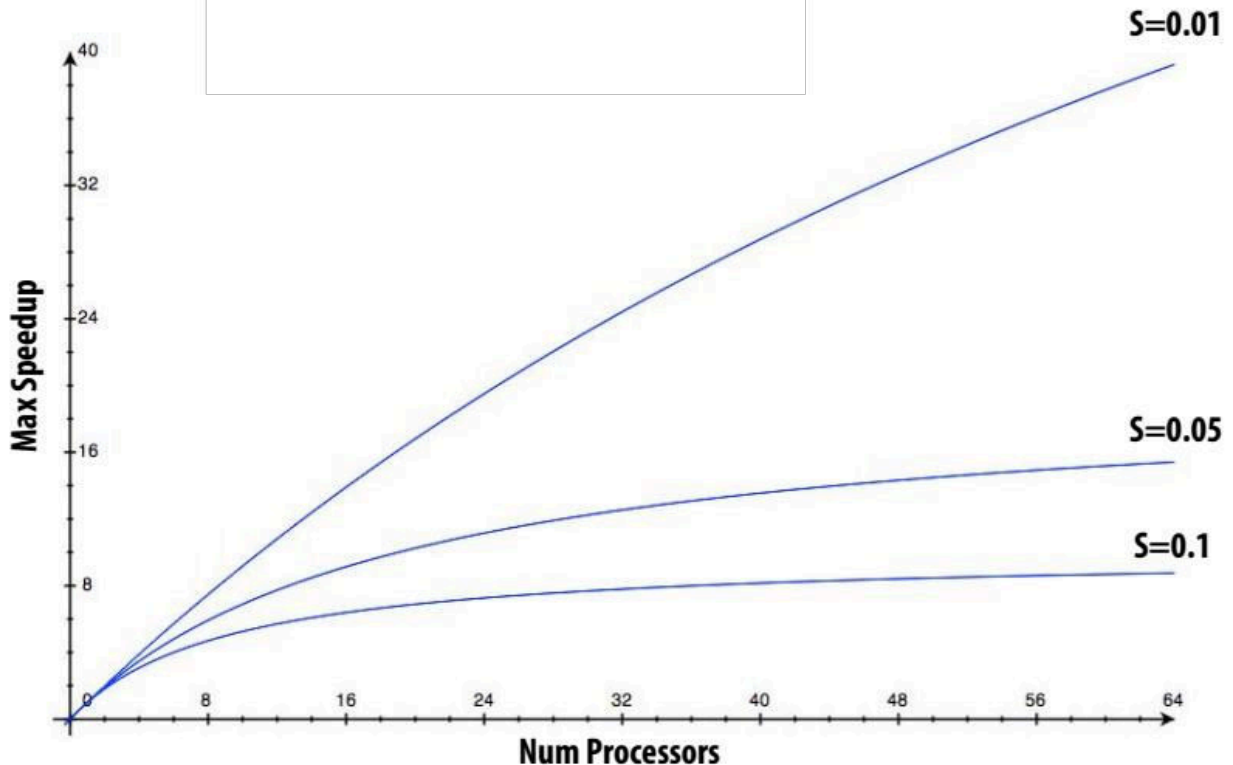
병렬처리에 의해서 50% 고속화 컴퓨터가 4대를 사용하면

$$\frac{1}{1 - 0.5 + (\frac{0.5}{4})} = \frac{1}{0.625} = 1.6$$

$$s = 1 - r$$

r 은 병렬처리 고속화 비율

$$\frac{1}{1 - r + (\frac{r}{p})}$$



A small serial region can limit speedup on a large parallel machine

- Summit supercomputer: 27,648 GPUs x (5,376 ALUs/GPU) = 148,635,648 ALUs
- Machine can perform 148 million single precision operations in parallel
- What is max speedup if 0.1% of application is serial?

병렬 컴퓨팅에서 작은 순차적 실행 부분이 전체 프로그램의 속도 향상을 제한함



1. 병렬 컴퓨팅의 한계:

- 병렬 컴퓨팅의 최대 속도 향상은 프로그램의 순차적 실행 부분에 의해 제한됨
- 예를 들어, 프로그램의 0.1%가 순차적으로 실행되어야 한다면, 병렬 컴퓨팅의 최대 속도 향상은 그 부분에 의해 제한됨.

예시:

- Summit 슈퍼컴퓨터는 148,635,648개의 ALU를 가지고 있으며, 이론적으로 148백만 개의 단일 정밀 연산을 병렬로 수행 가능
- 그러나 프로그램의 0.1%가 순차적으로 실행되어야 한다면, 최대 속도 향상은 1000배로 제한됨

병렬 프로그램을 작성할 때 순차적 실행 부분을 최소화하는 것이 중요

Decomposition

병렬 처리(Parallel Processing)는 하나의 문제를 해결하기 위해 여러 개의 처리 장치(CPU 코어, GPU 등)가 동시에 협력하여 작업을 수행하는 방식



- 병렬 처리를 효과적으로 수행하기 위한 첫 번째이자 **가장 중요한 단계 중 하나가 바로 Decomposition (분해)**
 - Decomposition은 크고 복잡한 문제를 병렬로 처리할 수 있는 작고 독립적인 여러 개의 하위 문제(Sub-problem) 또는 작업(Task)으로 나누는 과정
-
- 프로그램을 독립적인 작업으로 분해하는 책임은 누구에게 있나요?
 - 대부분의 경우: 프로그래머
 - 순차적 프로그램의 자동 분해(자동 병렬화)는 여전히 어려운 연구 문제(일반적인 경우 매우 어려움)
 - 컴파일러가 프로그램을 분석하고 의존성을 식별해야 함
 - ✓ 종속성이 데이터 종속적인 경우(컴파일 시점에 알 수 없음) 어떻게 해야 할까요?
 - **연구자들은 간단한 루프 중첩으로 소소한 성공을 거두었음.**
 - 복잡한 범용 코드를 위한 “마법의 병렬화 컴파일러”는 아직 달성되지 않았음

Assignment

- 작업자에게 작업 할당하기
 - “작업”을 해야 할 일로 생각하기
 - “작업자”란 무엇인가요? (threads, program instances, vector lanes, 등일 수 있음)
- 목표: 워크로드 균형(workload balance) 달성, 통신 비용(communication costs) 절감
- 정적(애플리케이션 실행 전) 또는 프로그램 실행 시 동적으로 수행 가능
- 분해(decomposition)는 프로그래머가 담당하는 경우가 많지만, 많은 언어/런타임이 할당(assignment)을 담당.

Decomposition이 문제의 병렬성을 추출하고, 여러 프로세서에 작업을 분배하여 로드 밸런싱(Load Balancing, 작업 부하 분산)을 개선하며, 통신 오버헤드(Communication Overhead, 처리 장치 간 데이터 교환에 걸리는 시간)를 최소화하는 데 중요한 역할

Decomposition의 주요 유형

1.Task Decomposition (작업 분해):

- 전체 문제를 기능적으로 독립적인 여러 작업으로 나눔.
- 각 작업은 다른 작업과 거의 또는 전혀 의존성이 없음.

2.Data Decomposition (데이터 분해):

- 문제를 해결하는 데 사용되는 데이터를 여러 부분으로 나누고, 각 처리 장치가 데이터의 특정 부분을 처리하도록 함.
- 동일한 연산이 대량의 데이터에 대해 반복될 때 효과적임.

Assignment examples in ISPC

```
export void ispc_sinx_interleaved(  
    uniform int N,  
    uniform int terms,  
    uniform float* x,  
    uniform float* result)  
{  
    // assumes N % programCount = 0  
    for (uniform int i=0; i<N; i+=programCount)  
    {  
        int idx = i + programIndex;  
        float value = x[idx];  
        float number = x[idx] * x[idx] * x[idx];  
        uniform int denom = 6; // 3!  
        uniform int sign = -1;  
  
        for (uniform int j=1; j<=terms; j++)  
        {  
            value += sign * number / denom;  
            number *= x[idx] * x[idx];  
            denom *= (2*j+2) * (2*j+3);  
            sign *= -1;  
        }  
        result[i] = value;  
    }  
}
```

- 루프 반복을 통한 작업 분해
- 프로그래머 관리 할당:
 - ✓ 정적 할당(Static assignment)
 - ✓ 인터리브 방식으로 ISPC 프로그램 인스턴스에 반복을 할당

```
export void ispc_sinx_foreach(  
    uniform int N,  
    uniform int terms,  
    uniform float* x,  
    uniform float* result)  
{  
    foreach (i = 0 ... N)  
    {  
        float value = x[i];  
        float number = x[i] * x[i] * x[i];  
        uniform int denom = 6; // 3!  
        uniform int sign = -1;  
  
        for (uniform int j=1; j<=terms; j++)  
        {  
            value += sign * number / denom;  
            number *= x[i] * x[i];  
            denom *= (2*j+2) * (2*j+3);  
            sign *= -1;  
        }  
        result[i] = value;  
    }  
}
```

- loop iteration foreach 구조에 의한 작업 분해(Decomposition)는 독립적인 작업을 시스템에 노출.
- ISPC 프로그램 인스턴스에 대한 반복(작업) 할당 시스템 관리
- ((추상화는 동적 할당을 위한 여지를 남기지만 현재 ISPC 구현은 정적))

Example 2: static assignment using C++11 threads

```
void my_thread_start(int N, int terms, float* x, float* results) {
    sinx(N, terms, x, result); // do work
}

void parallel_sinx(int N, int terms, float* x, float* result) {

    int half = N/2.

    // launch thread to do work on first half of array
    std::thread t1(my_thread_start, half, terms, x, result);

    // do work on second half of array in main thread
    sinx(N - half, terms, x + half, result + half);

    t1.join();
}
```

루프 반복을 통한 작업 분해

프로그래머가 관리하는 정적 할당

이 프로그램은 루프 반복을 블록화된 방식으로 스레드에 할당
(배열의 전반부는 스폰된 스레드에 할당, 후반부는 메인 스레드에 할당).

병렬 처리에서 **할당(Assignment)**은 분해된 하위 작업 또는 데이터를 어떤 처리 장치(스레드, 프로세스 등)가 처리할 것인지 결정하는 과정.

정적 할당 (Static Assignment)이란?

- 정적 할당은 이러한 할당이 프로그램 실행 전에 미리 결정되는 방식을 의미. 즉, 컴파일 시점이나 프로그램 시작 시점에 각 스레드가 담당할 작업의 범위가 고정되는 것.
- 동적 할당(Dynamic Assignment)은 프로그램 실행 중에 시스템의 부하 상태나 작업 완료 상황 등을 고려하여 실시간으로 작업 할당을 변경하는 방식

Dynamic assignment using ISPC tasks

```
void foo(uniform float* input,  
        uniform float* output,  
        uniform int N)  
{  
    // create a bunch of tasks  
    launch[100] my_ispc_task(input, output, N);  
}
```

ISPC 런타임(프로그래머에게 보이지 않음) 이스레드 풀
(thread pool)의 워커 스레드(worker threads)에 작업()을 할당.

Next task ptr



List of tasks:

task 0	task 1	task 2	task 3	task 4	...	task 99
--------	--------	--------	--------	--------	-----	---------

스레드에 작업 할당 구현: 현재 작업을 완료한 후 작업자 스레드는 목록을 검사하고 완료되지 않은 다음 작업을 스스로 할당

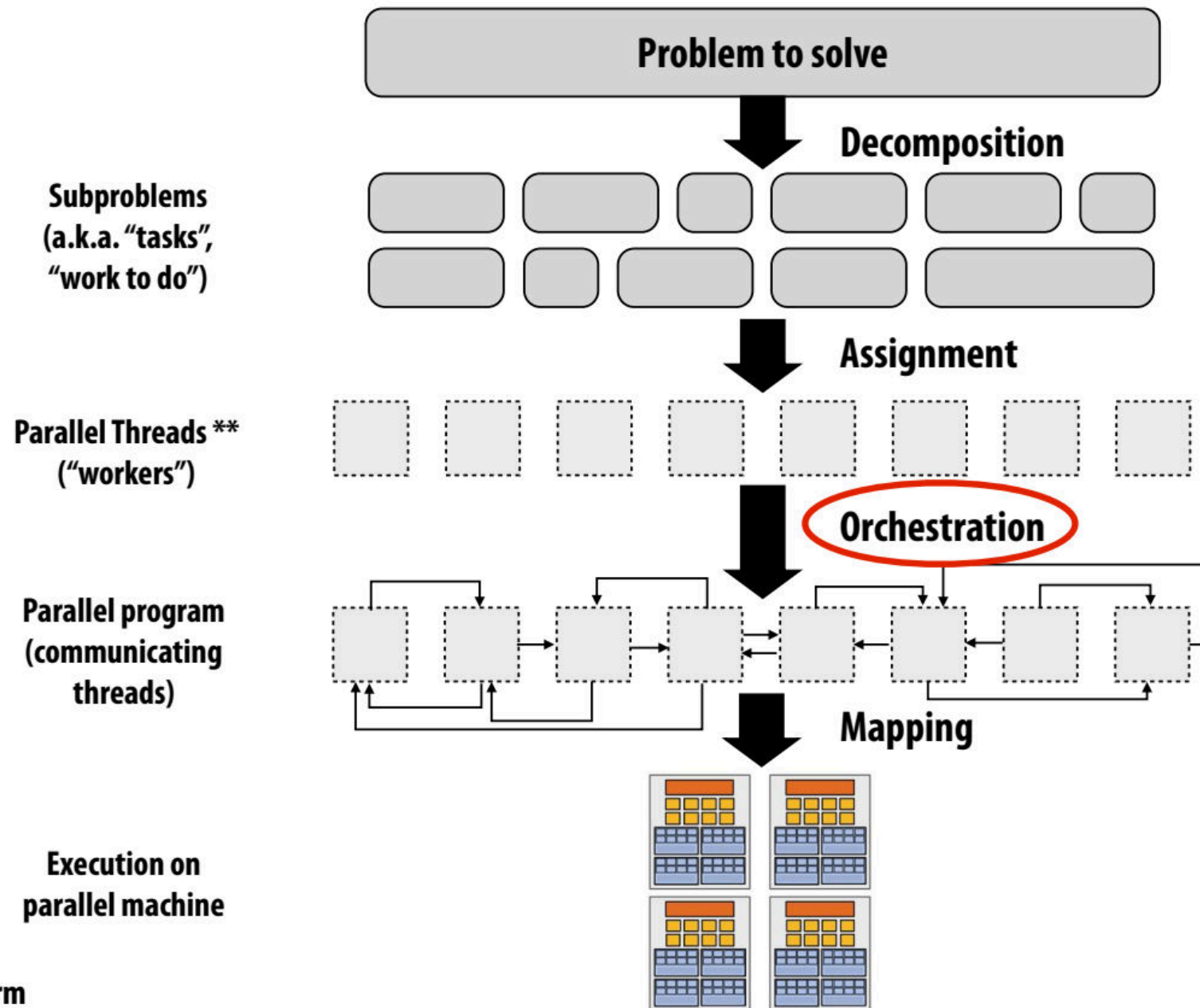
Worker
thread 0

Worker
thread 1

Worker
thread 2

Worker
thread 3

Orchestration

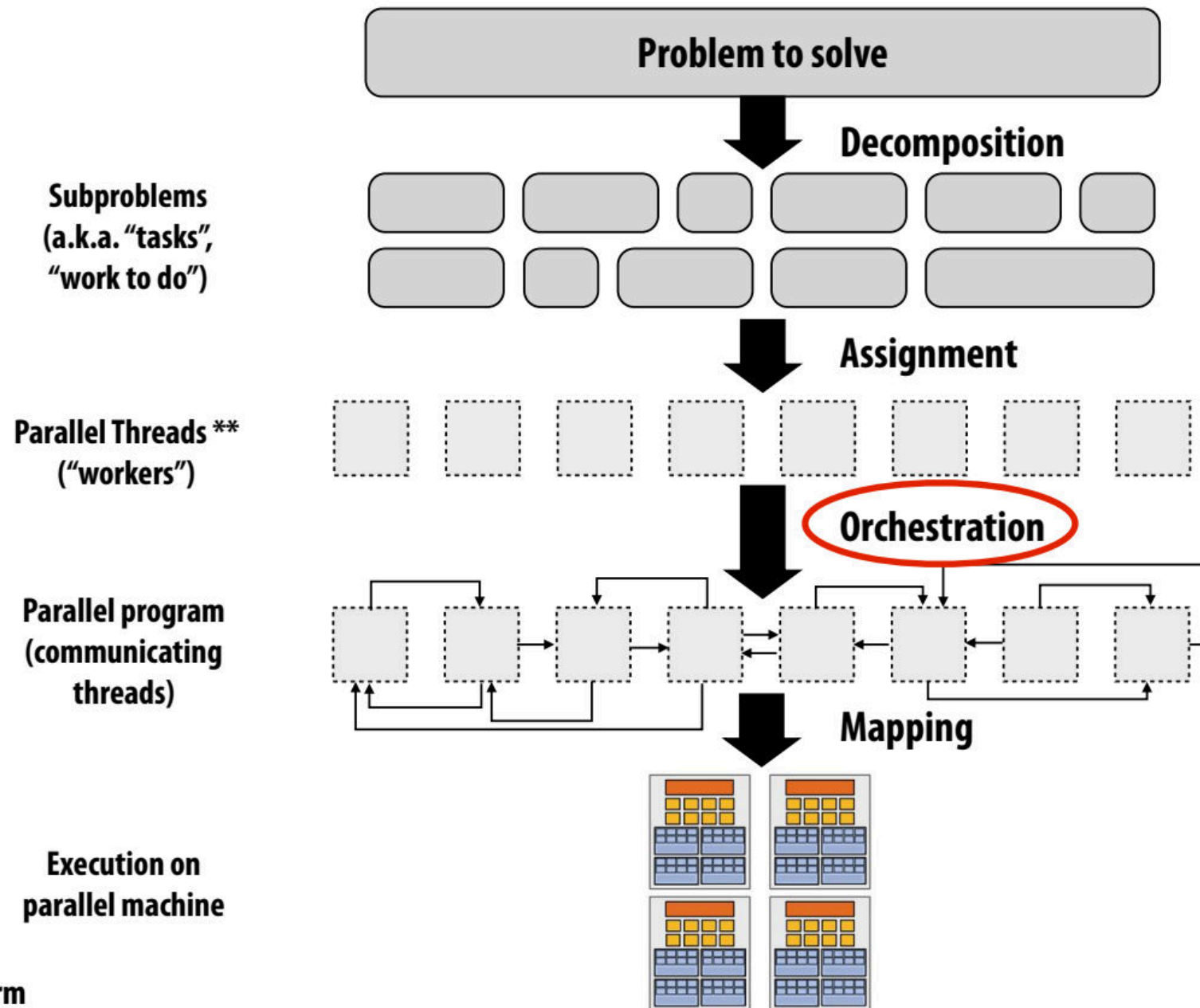


** I had to pick a term

Orchestration(오케스트레이션)

- Orchestration (오케스트레이션)은 분해된 여러 개의 하위 작업(Task)들이나 데이터 조각들이 **올바른 순서와 방식으로 실행되도록 조정하고 관리하는 과정**
- Decomposition(분해)이 문제를 작게 나누는 과정이라면, Orchestration은 이렇게 나누어진 작은 조각들이 전체 문제를 해결하기 위해 **어떻게 협력하고 상호작용해야 하는지를 설계하고 제어하는 과정**
- Involves:
 - 커뮤니케이션 구조화
 - 필요한 경우 의존성 보존을 위한 동기화 추가
 - 메모리 내 데이터 구조 구성
 - 작업 스케줄링
- Goals: 통신/동기화 비용 절감, 데이터 참조의 로컬리티 보존, 오버헤드 감소 등
- 머신 세부 사항은 이러한 결정에 많은 영향을 미침.
 - 동기화 비용이 많이 드는 경우 프로그래머는 동기화를 더 드물게 사용할 수 있음

Orchestration



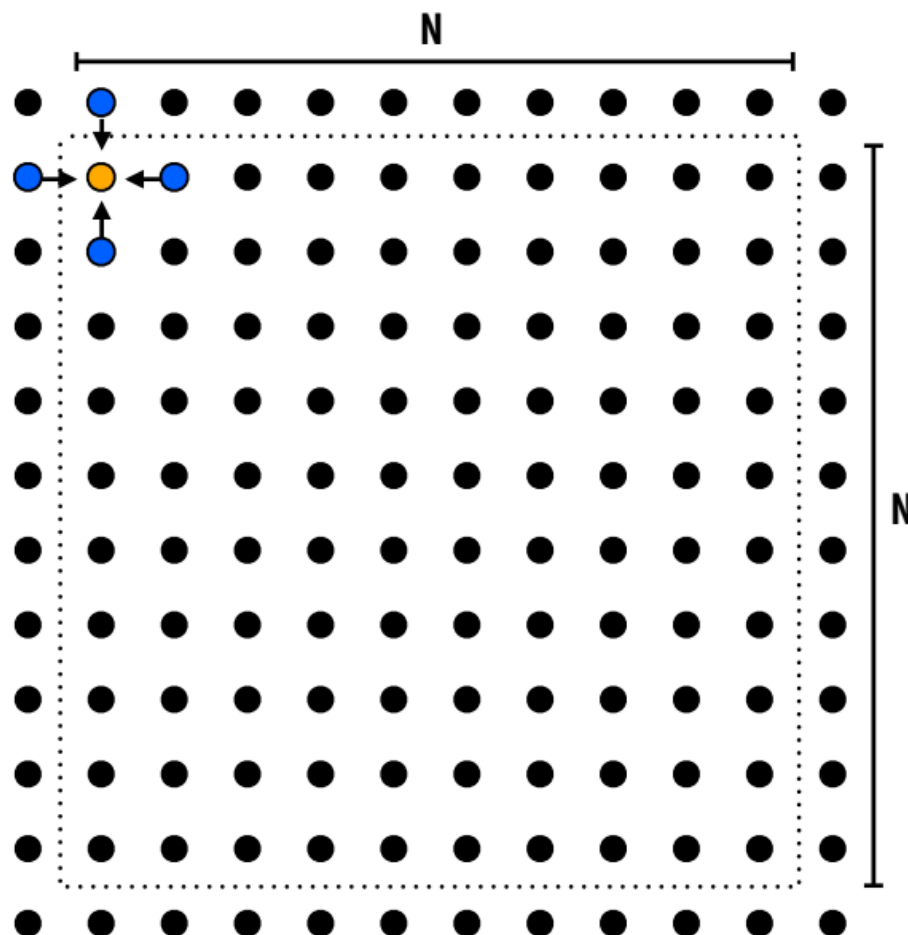
** I had to pick a term

Mapping to hardware

- 하드웨어 실행 유닛(hardware execution unit)에 “threads”(“workers”)를 매핑하기
- 예시 1: 운영체제(OS)별 매핑
 - 예: CPU 코어의 HW 실행 컨텍스트에 스레드 매핑
- 예 2: 컴파일러에 의한 매핑
 - ISPC 프로그램 인스턴스를 벡터 명령어 레인에 매핑하기
- 예 3: 하드웨어에 의한 매핑
 - CUDA 스레드 블록(CUDA thread blocks)을 GPU 코어에 매핑하기(추후 강의에서 설명)
- 많은 흥미로운 매핑 결정
 - 관련 스레드(협력 스레드)를 동일한 코어에 배치(로컬리티, 데이터 공유 최대화, 통신/동기화 비용 최소화)
 - 관련 없는 스레드를 동일한 코어에 배치(하나는 대역폭이 제한되고 다른 하나는 컴퓨팅이 제한될 수 있음)하여 머신을 보다 효율적으로 사용

A parallel programming example

- 문제: $(N+2) \times (N+2)$ 그리드에서 편미분 방정식(PDE) 풀기
- 솔루션은 반복 알고리즘을 사용:
 - 수렴할 때까지 그리드에 대해 가우스-사이델 스위프 (Gauss-Seidel sweeps over) 수행



$$A[i,j] = 0.2 * (A[i,j] + A[i,j-1] + A[i-1,j] + A[i,j+1] + A[i+1,j]);$$

The Gauss-Seidel Method

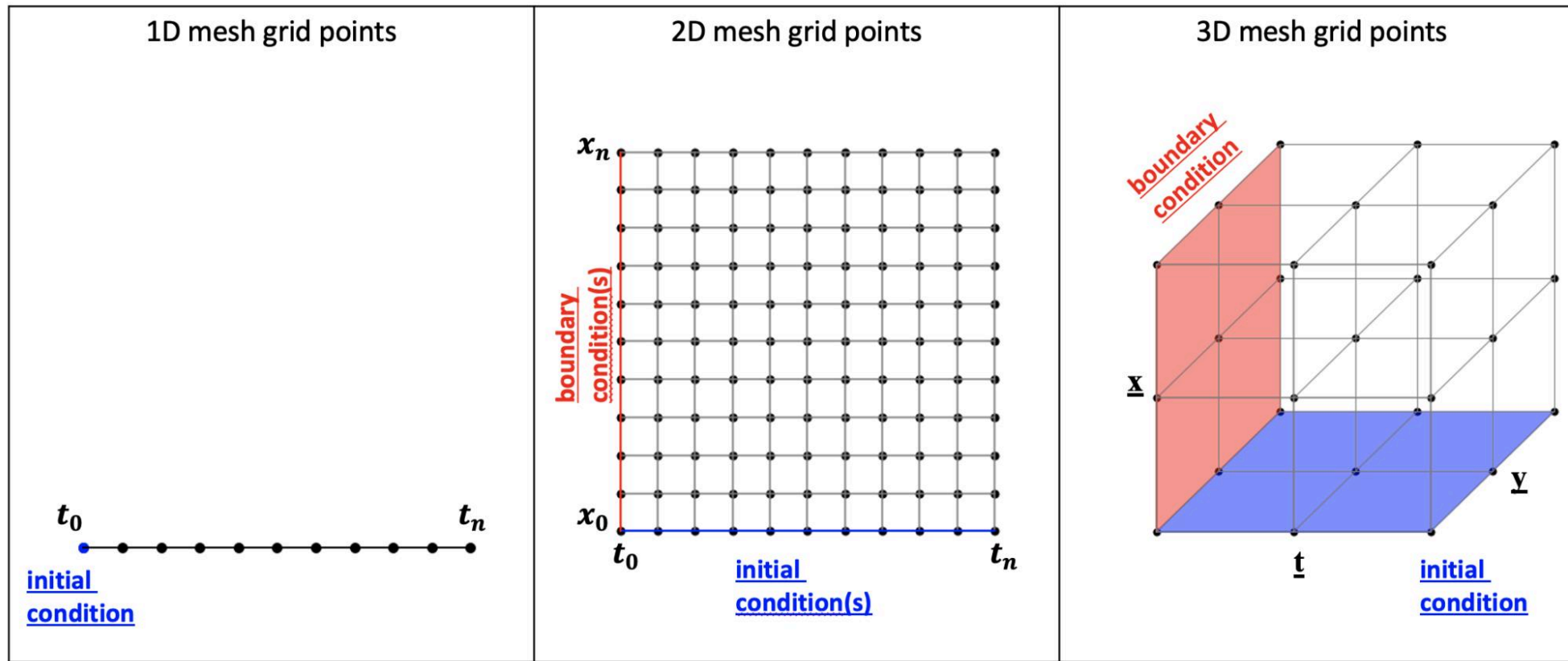
- Recall the discretized equation

$$T_{i+1,j} + T_{i-1,j} + T_{i,j+1} + T_{i,j-1} - 4T_{i,j} = 0$$

- This can be rewritten as

$$T_{i,j} = \frac{T_{i+1,j} + T_{i-1,j} + T_{i,j+1} + T_{i,j-1}}{4}$$

- For the Gauss-Seidel Method, this equation is solved iteratively for all interior nodes until a pre-specified tolerance is met.



- **2D-grid based solver**는 물리 시뮬레이션, 이미지 처리, 유한 차분법(Finite Difference Method)을 사용한 편미분 방정식 해법 등 많은 분야에서 나타나는 문제를 해결하기 위해 사용됨.
- 이러한 문제들은 2차원 공간을 격자로 나누고, 각 격자 점(grid point)에서의 값을 주변 격자 점들의 값을 이용하여 계산하는 특징을 가짐

- SOR (Successive Over-Relaxation) 기법이란?

- SOR 기법은 주로 편미분 방정식(PDE)을 이산화(discretization)하여 얻어지는 **대규모 선형 연립방정식** $Ax=b$ 를 푸는 데 사용되는 **반복법(iterative method)** 중 하나.
- 특히 라플라스 방정식이나 푸아송 방정식과 같은 타원형 편미분 방정식 문제에서 자주 등장
- 기본 아이디어는 각 미지수 x_i 를 업데이트할 때, 해당 방정식에서 다른 변수들의 최신 값(같은 반복 단계에서 이미 계산된 값 포함)과 이전 반복 단계의 값을 이용하여 새로운 값을 계산하는 것입니다. 여기에 수렴 속도를 높이기 위해 이완 계수(relaxation factor) ω 를 사용.

$$x_i^{(k+1)} = (1 - \omega)x_i^{(k)} + \frac{\omega}{a_{ii}} \left(b_i - \sum_{j < i} a_{ij}x_j^{(k+1)} - \sum_{j > i} a_{ij}x_j^{(k)} \right)$$

$x_j^{(k+1)}$ ($j < i$)항 때문에 $x_i^{(k+1)}$ 계산은 $x_1^{(k+1)}, x_2^{(k+1)}, \dots, x_{i-1}^{(k+1)}$ 이 순차적으로 계산되어야 함.

이것이 **데이터 의존성(data dependency)**이며, 표준 SOR 기법을 직접 병렬화하기 어렵게 만드는 주된 이유

- Red-Black SOR: 병렬화를 위한 아이디어

- Red-Black SOR 기법은 이러한 데이터 의존성을 깨뜨려 병렬 처리를 가능하게 하는 방법으로 격자(grid) 기반 문제 (예: 유한 차분법)에서 특히 효과적

아이디어:

격자점(Grid Points) 색칠:

- 문제의 격자점을 마치 체스판처럼 두 가지 색깔, 즉 'Red'와 'Black'으로 번갈아 색칠.
- 2차원 격자의 경우, 좌표 (i,j) 에 대해 $(i+j)$ 가 짝수이면 Red, 홀수이면 Black (또는 그 반대)으로 지정할 수 있음.

핵심 속성:

- Red 격자점의 직접적인 이웃(상하좌우)은 모두 Black 격자점이고, Black 격자점의 직접적인 이웃은 모두 Red 격자점이라는 것.

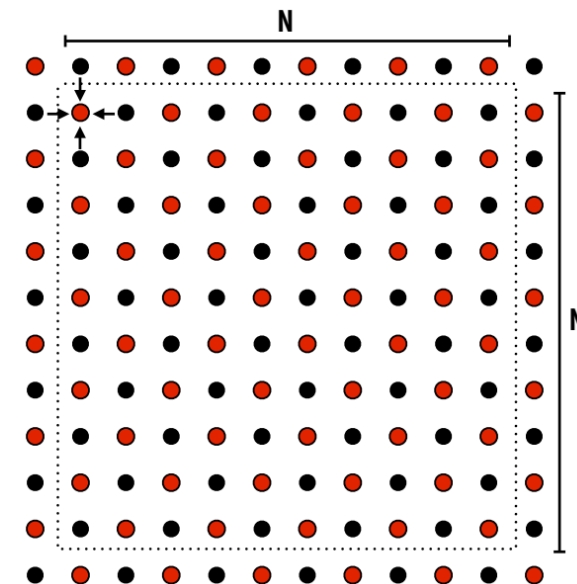
병렬 업데이트:

- Red 점 업데이트:

- 모든 Red 점들을 업데이트. Red 점 $x_{i,j}$ 의 SOR 업데이트는 이웃한 Black 점들의 값에만 의존.
- 따라서 모든 Red 점들은 서로 독립적으로, **동시에 (병렬적으로)** 업데이트될 수 있음. 이때 사용되는 Black 점들의 값은 이전 반복 단계(k)의 값.

- Black 점 업데이트:

- 모든 Red 점들의 업데이트가 완료되면, 모든 Black 점들을 업데이트.
- Black 점 $x_{i,j}$ 의 업데이트는 이웃한 Red 점들의 값에만 의존. 따라서 모든 Black 점들도 서로 독립적으로, **동시에 (병렬적으로)** 업데이트될 수 있음.
- 이때 사용되는 Red 점들의 값은 방금 전 단계에서 계산된 새로운 값(k+1 단계 값).

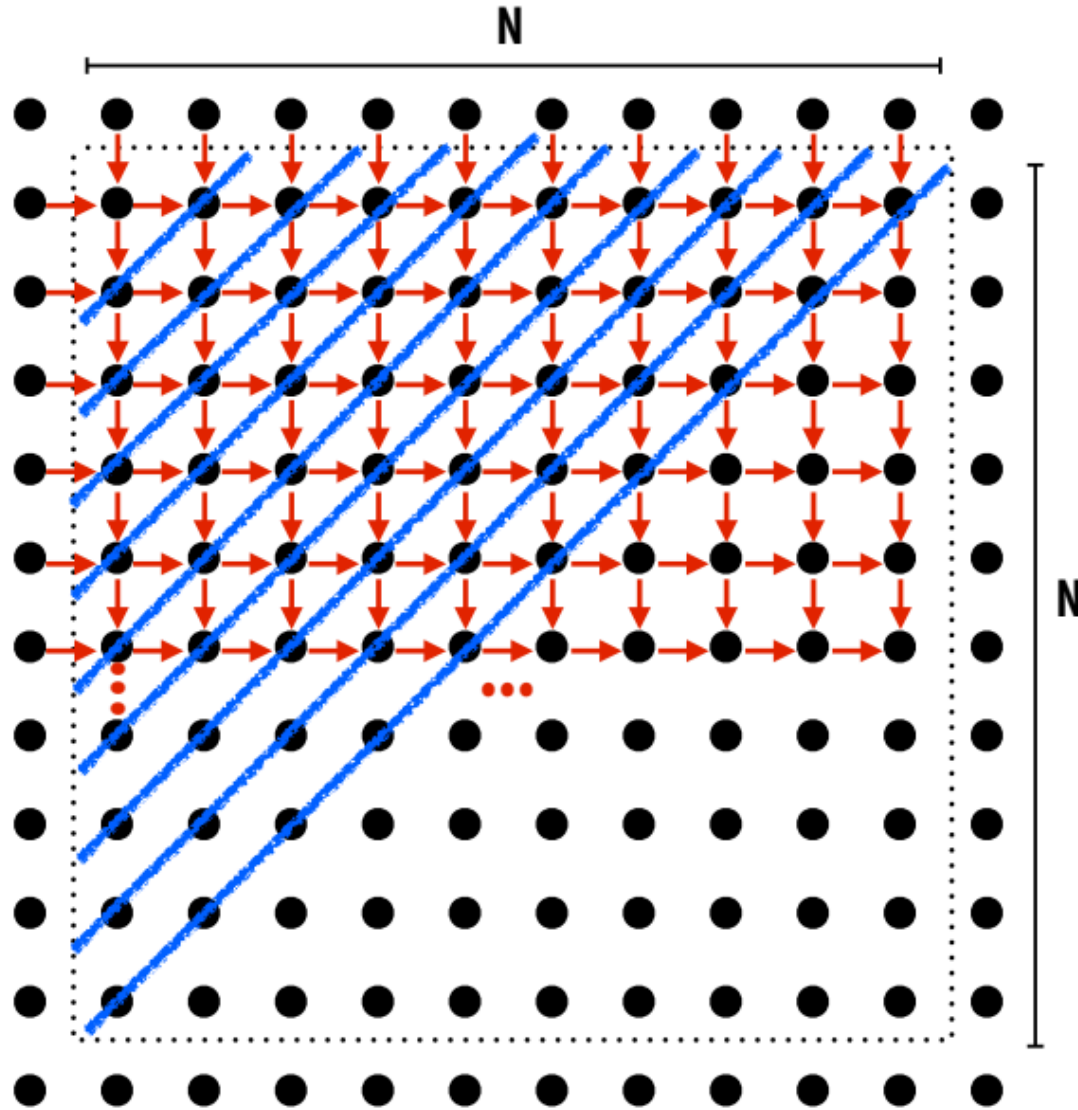


Grid solver algorithm: find the dependencies

Pseudocode for sequential algorithm is provided below

```
const int n;  
float* A;           // assume allocated for grid of N+2 x N+2 elements  
  
void solve(float* A) {  
    float diff, prev;  
    bool done = false;  
  
    while (!done) {           // outermost loop: iterations  
        diff = 0.f;  
        for (int i=1; i<n i++) {           // iterate over non-border points of grid  
            for (int j=1; j<n; j++) {  
                prev = A[i,j];  
                A[i,j] = 0.2f * (A[i,j] + A[i,j-1] + A[i-1,j] +  
                                A[i,j+1] + A[i+1,j]);  
                diff += fabs(A[i,j] - prev); // compute amount of change  
            }  
        }  
  
        if (diff/(n*n) < TOLERANCE) // quit if converged  
            done = true;  
    }  
}
```

Step 1: 종속성 식별(identify dependencies) → (problem decomposition phase)



대각선을 따라 독립적인 작업이 있음!

Good(좋은점): 병렬성이 존재!

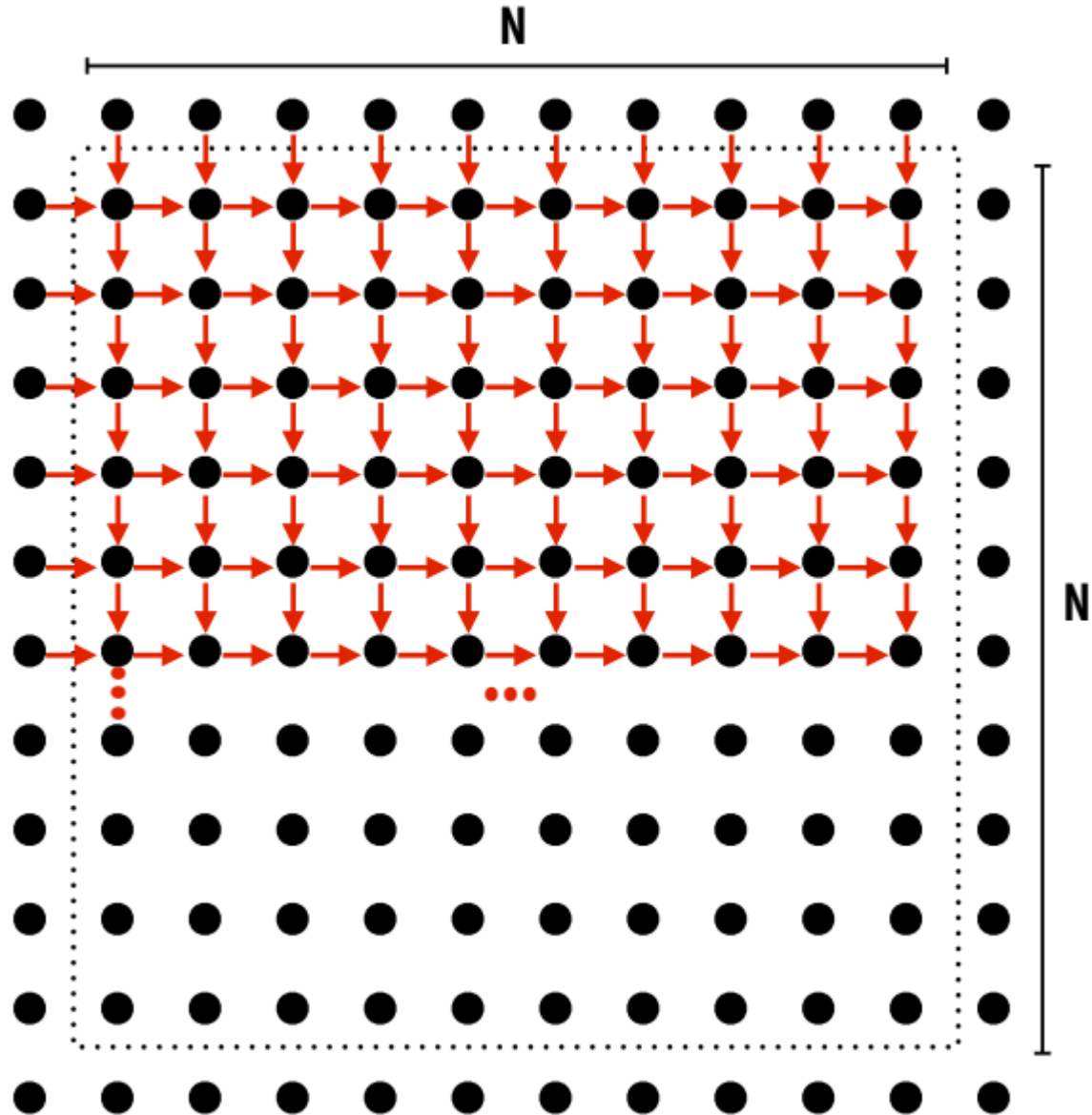
가능한 구현 전략:

1. 대각선의 그리드 셀을 작업으로 분할.
2. 값을 병렬로 업데이트
3. 완료되면 다음 대각선으로 이동

Bad(나쁜점): 독립적인 작업을 활용하기 어려움

- 계산의 시작과 끝에서 병렬 처리가 많지 않음.
- 잦은 동기화(각 대각선 완료 후)

Step 1: 종속성 식별(identify dependencies) → (problem decomposition phase)

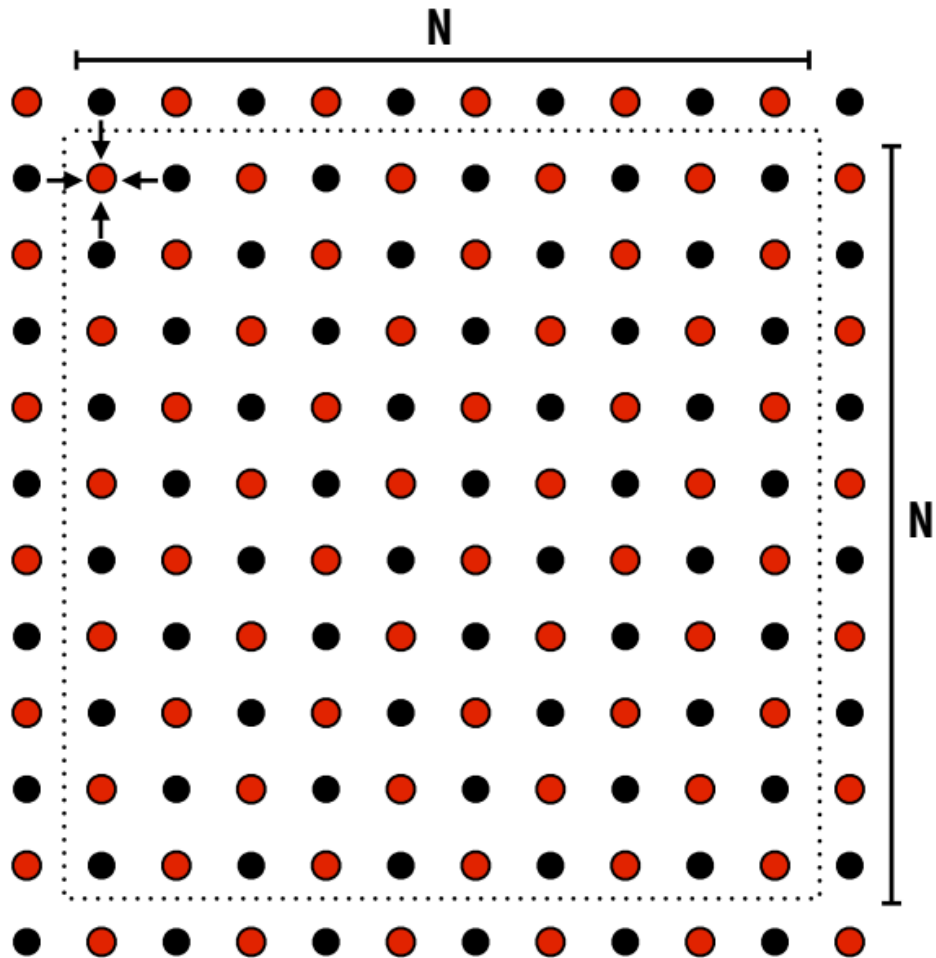


각 행 요소는 왼쪽 요소에 종속.
각 행은 이전 행에 종속.

참고: 이 슬라이드에 표시된 의존성은 solver의 한번 반복(in one iteration of the "while not done" loop)에서 그리드 요소 데이터 의존성.

New approach: reorder grid cell update via red-black coloring

그리드 순회 재정렬(Reorder grid traversal): 빨간색-검정색 채색(red-black coloring)



- 모든 **red cells**를 병렬로 업데이트
- **red cells** 업데이트가 완료되면, 모든 검정색 셀을 병렬로 업데이트. (**red cells**에 대한 의존성 반영)
- 수렴할 때까지 반복

- SOR (Successive Over-Relaxation) 기법이란?

- SOR 기법은 주로 편미분 방정식(PDE)을 이산화(discretization)하여 얻어지는 **대규모 선형 연립방정식** $Ax=b$ 를 푸는 데 사용되는 **반복법(iterative method)** 중 하나.
- 특히 라플라스 방정식이나 푸아송 방정식과 같은 타원형 편미분 방정식 문제에서 자주 등장
- 기본 아이디어는 각 미지수 x_i 를 업데이트할 때, 해당 방정식에서 다른 변수들의 최신 값(같은 반복 단계에서 이미 계산된 값 포함)과 이전 반복 단계의 값을 이용하여 새로운 값을 계산하는 것입니다. 여기에 수렴 속도를 높이기 위해 이완 계수(relaxation factor) ω 를 사용.

$$x_i^{(k+1)} = (1 - \omega)x_i^{(k)} + \frac{\omega}{a_{ii}} \left(b_i - \sum_{j < i} a_{ij}x_j^{(k+1)} - \sum_{j > i} a_{ij}x_j^{(k)} \right)$$

$x_j^{(k+1)}$ ($j < i$)항 때문에 $x_i^{(k+1)}$ 계산은 $x_1^{(k+1)}, x_2^{(k+1)}, \dots, x_{i-1}^{(k+1)}$ 이 순차적으로 계산되어야 함.

이것이 **데이터 의존성(data dependency)**이며, 표준 SOR 기법을 직접 병렬화하기 어렵게 만드는 주된 이유

- Red-Black SOR: 병렬화를 위한 아이디어

- Red-Black SOR 기법은 이러한 데이터 의존성을 깨뜨려 병렬 처리를 가능하게 하는 방법으로 격자(grid) 기반 문제 (예: 유한 차분법)에서 특히 효과적

아이디어:

격자점(Grid Points) 색칠:

- 문제의 격자점을 마치 체스판처럼 두 가지 색깔, 즉 'Red'와 'Black'으로 번갈아 색칠.
- 2차원 격자의 경우, 좌표 (i,j) 에 대해 $(i+j)$ 가 짝수이면 Red, 홀수이면 Black (또는 그 반대)으로 지정할 수 있음.

핵심 속성:

- Red 격자점의 직접적인 이웃(상하좌우)은 모두 Black 격자점이고, Black 격자점의 직접적인 이웃은 모두 Red 격자점이라는 것.

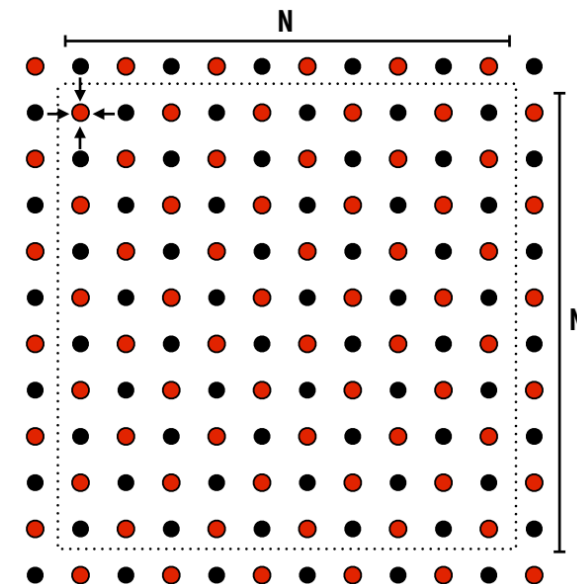
병렬 업데이트:

- Red 점 업데이트:

- 모든 Red 점들을 업데이트. Red 점 $x_{i,j}$ 의 SOR 업데이트는 이웃한 Black 점들의 값에만 의존.
- 따라서 모든 Red 점들은 서로 독립적으로, **동시에 (병렬적으로)** 업데이트될 수 있음. 이때 사용되는 Black 점들의 값은 이전 반복 단계(k)의 값.

- Black 점 업데이트:

- 모든 Red 점들의 업데이트가 완료되면, 모든 Black 점들을 업데이트.
- Black 점 $x_{i,j}$ 의 업데이트는 이웃한 Red 점들의 값에만 의존. 따라서 모든 Black 점들도 서로 독립적으로, 동시에 (병렬적으로) 업데이트될 수 있음.
- 이때 사용되는 Red 점들의 값은 방금 전 단계에서 계산된 새로운 값(k+1 단계 값).



Possible assignments of work to processors

그리드 순회 재정렬(Reorder grid traversal): 빨간색-검정색 채색(red-black coloring)

1. 블록 할당 (Blocked Assignment)

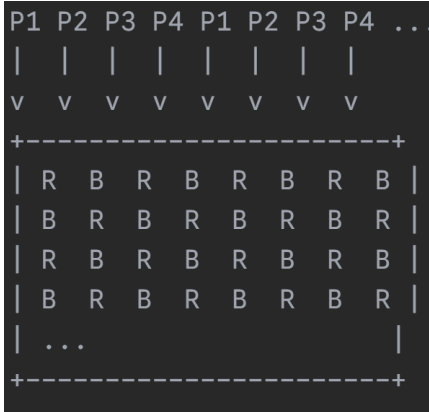
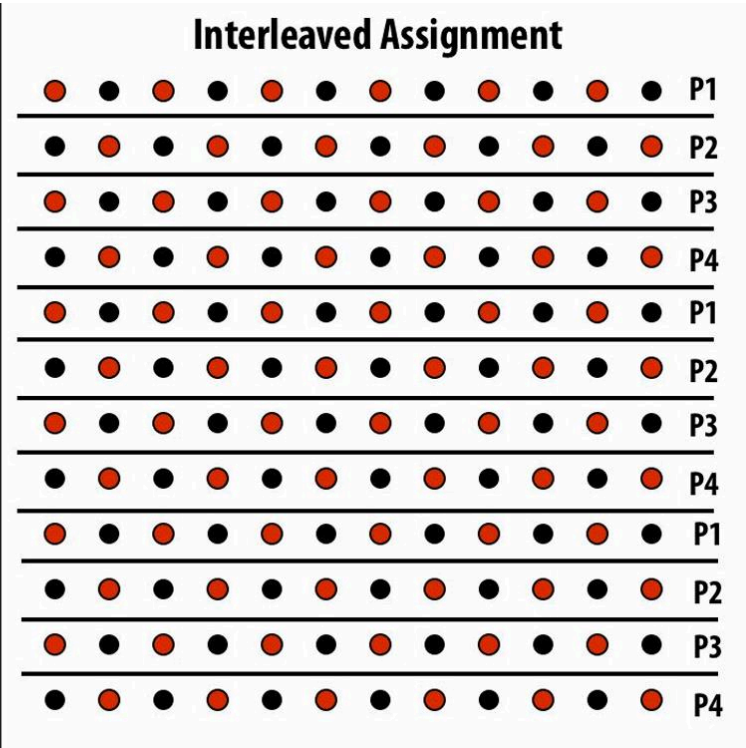
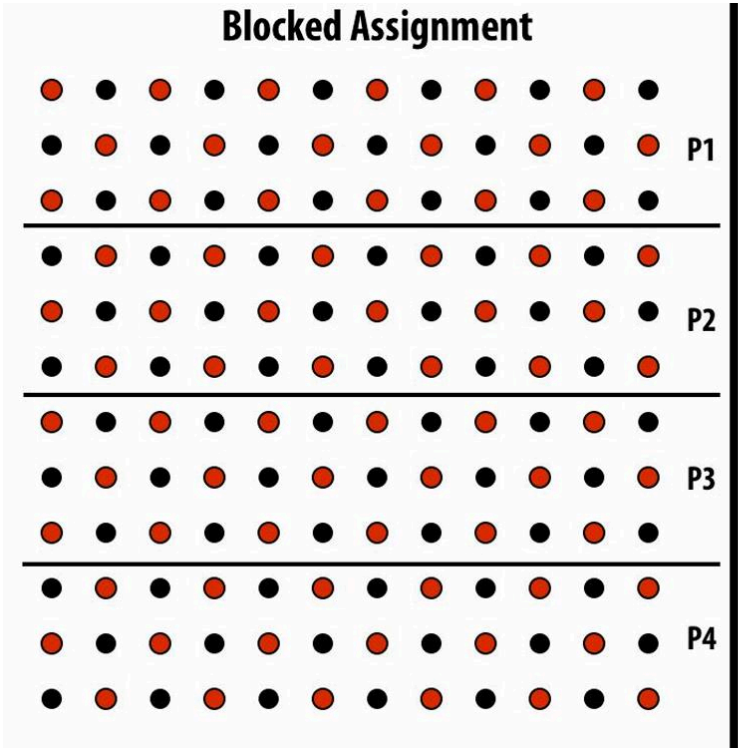
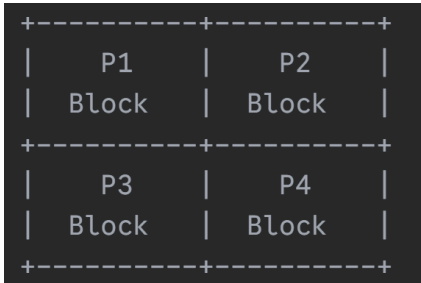
•개념: 전체 작업 공간(여기서는 2D 격자)을 **연속적인 큰 덩어리(block)**로 나눈 다음, 각 덩어리를 하나의 프로세서에 할당하는 방식.

- 데이터 지역성 (Data Locality): 각 프로세서가 담당하는 데이터가 메모리 상에서 비교적 가까이 모여 있을 가능성이 높습니다. 이는 캐시(cache) 활용에 유리할 수 있음.
- 통신: 주로 블록의 경계(edge)에 있는 데이터를 처리할 때 다른 프로세서와의 통신이 필요

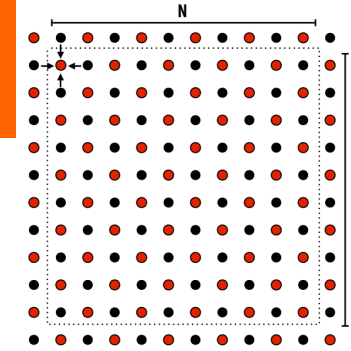
2. 인터리빙 할당 (Interleaved Assignment)

•개념: 작업 단위(여기서는 격자의 행 또는 열)를 번갈아 가면서(interleave) 여러 프로세서에 할당하는 방식입니다. 마치 카드를 나누어 주듯이 순환적으로 할당한다고 해서 **순환 할당 (Cyclic Assignment)**이라고도 함.

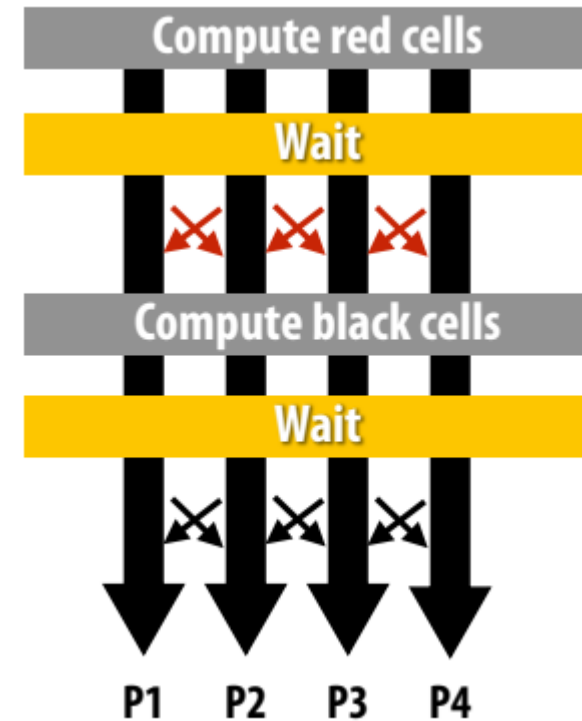
- 부하 균형 (Load Balancing): 격자 내 특정 영역에 계산 부하가 몰려 있더라도, 작업을 잘게 나누어 분산시키므로 프로세서 간의 부하를 비교적 균등하게 유지하는 데 유리
- 데이터 지역성: 각 프로세서가 담당하는 데이터가 메모리 상에서 흩어져 있을 가능성이 높아 캐시 활용에는 불리.
- 통신: 인접한 데이터를 처리하기 위해 더 많은 또는 더 빈번한 통신이 필요할 수 있음.



프로그램의 의존성 고려

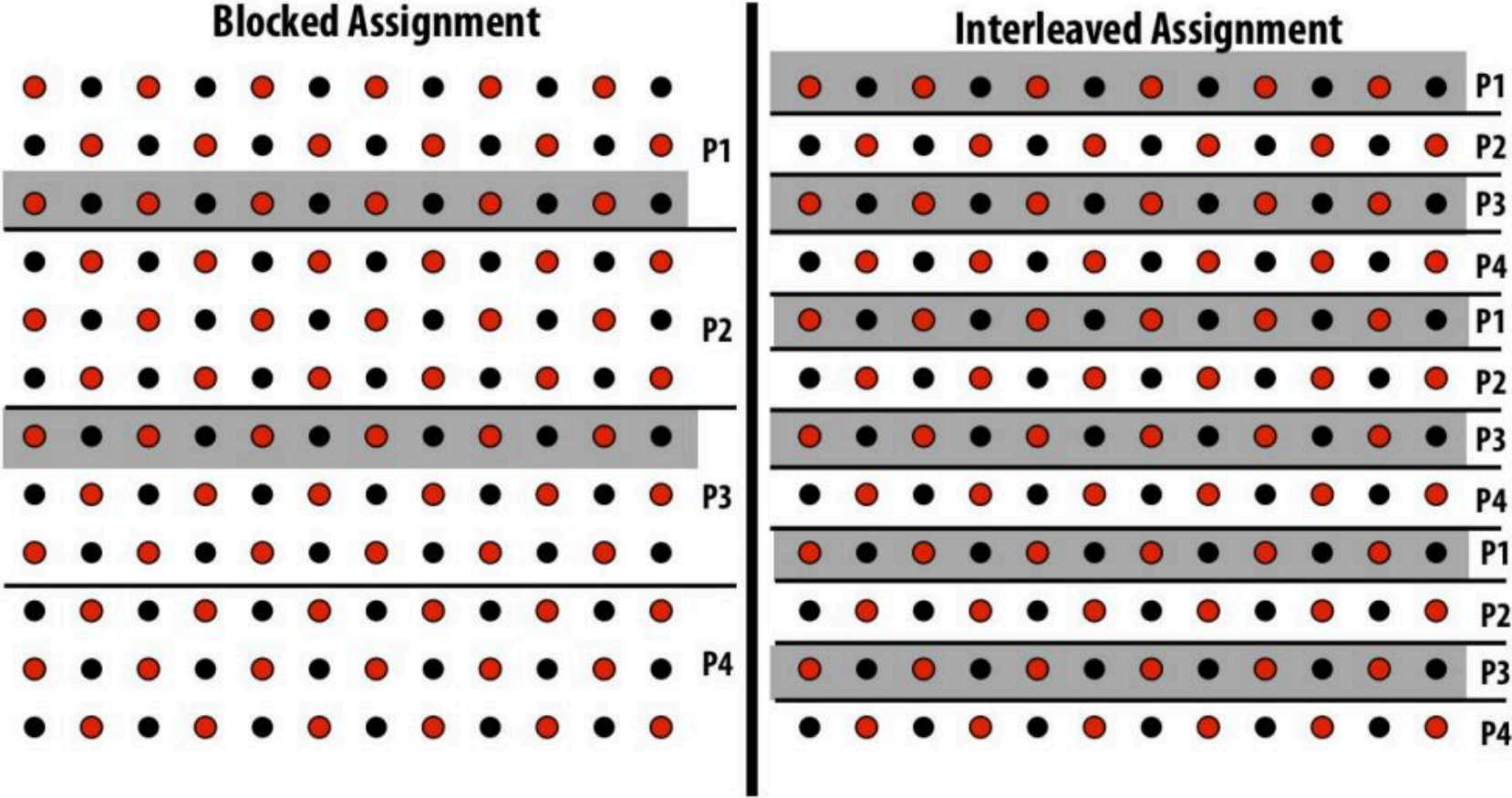


1. 적색 셀 업데이트를 병렬로 수행.
2. 모든 프로세서가 업데이트가 완료될 때까지 대기
3. 업데이트된 적색 셀을 다른 프로세서에 전달
4. 블랙 셀 업데이트를 병렬로 수행.
5. 모든 프로세서가 업데이트가 완료될 때까지 대기
6. 업데이트된 블랙 셀을 다른 프로세서에 전달
7. 반복



Communication resulting from assignment

그리드 순회 재정렬(Reorder grid traversal): 빨간색-검정색 채색(red-black coloring)



= 매 반복마다 P2로 전송해야 하는 데이터
Blocked assignment을 사용하면 프로세서 간에 통신해야 하는 데이터가 줄어듬(감소).

프로그램 작성에 대해 생각하는 두 가지 방법

- **Data parallel thinking**
- **SPMD / shared address space**

Data-parallel expression of solver

Data-parallel expression of solver

Note: 의사 코드를 단순화하기 위해: red-cell update만 표시

데이터 병렬 모델: 대규모 데이터 집합에 대해 동일한 연산을 동시에 적용하는 방식에 초점을 맞춘 병렬 프로그래밍 패러다임으로, 프로그래머는 주로 데이터와 그 데이터에 적용할 연산을 정의하며, 시스템(컴파일러, 런타임)이 이 연산을 여러 처리 장치에 분산시켜 병렬로 실행하는 것을 담당

```
const int n;  
float* A = allocate(n+2, n+2));
```

// 격자 할당

```
void solve(float* A) {
```

```
    bool done = false;
```

```
    float diff = 0.f;
```

```
    while (!done) {
```

```
        for_all (red cells (i,j)) {
```

// --- Red 셀 업데이트 (데이터 병렬) ---

```
            float prev = A[i,j];
```

```
            A[i,j] = 0.2f * (A[i-1,j] + A[i,j-1] + A[i,j] +  
                           A[i+1,j] + A[i,j+1]);
```

```
            reduceAdd(diff, abs(A[i,j] - prev));
```

```
        }
```

```
    if (diff/(n*n) < TOLERANCE)
```

```
        done = true;
```

```
    }
```

```
}
```

Assignment: ???

할당

Decomposition: 개별 그리드 요소를 처리하는 것은 독립적인 작업으로 구성.

for_all (red cells (i,j))는 지정된 데이터 요소 집합(모든 Red 셀)에 대해 그 안에 있는 코드 블록({...})을 병렬적으로 실행하라는 의미. 즉, 이론적으로는 모든 Red 셀의 업데이트 계산이 동시에 일어날 수 있음.

Orchestration: 시스템에서 처리
(builtin communication primitive: reduceAdd)

Orchestration: 시스템에서 처리
for_all block의 끝은 순차 제어로 돌아가기 전에 모든 workers들을 암시적으로 대기

Shared address space(with SPMD threads) expression of solver

공유 주소 공간
(Shared Address Space, SAS) 모델



컴퓨터 시스템의 메인 메모리(RAM)를 모든 프로세서(코어)나 스레드가 함께 공유하는 환경

- 공유 데이터:

- 여러 스레드가 A라는 배열이나 diff라는 변수처럼 **동일한 메모리 위치**에 있는 데이터에 직접 접근(읽기/쓰기)할 수 있음. 마치 여러 사람이 **하나의 큰칠판**에 같이 글을 쓰거나 읽을 수 있는 것과 비슷.

- 통신: 스레드 간의 데이터 교환은 이 공유 메모리에 값을 쓰고 읽는 방식으로 이루어짐

- 주의점:

- 여러 스레드가 동시에 같은 데이터를 수정하려고 하면 ****경쟁 상태(race condition)****가 발생하여 데이터가 엉망이 될 수 있음. 따라서 **동기화(synchronization)** 메커니즘이 필수적.

SPMD(Single Program, Multiple Data)
실행 모델



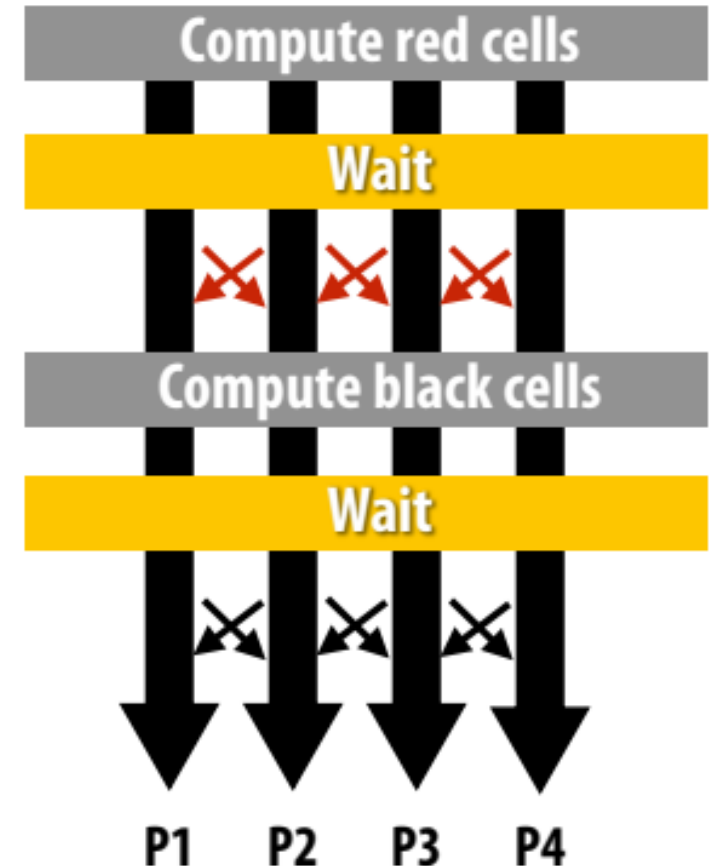
SPMD는 여러 스레드가 **동일한 프로그램 코드**를 실행하는 방식

차별화: 각 스레드는 자신만의 **고유 ID (threadId)**를 가지며, 이 ID를 사용해 자기가 처리해야 할 데이터 영역(예: 격자의 특정 행 범위)을 계산하거나 코드 내에서 분기하여 다른 동작을 할 수 있음

Shared address space expression of solver

SPMD execution model

- 프로그래머가 동기화를 담당.
- Common synchronization primitives :
 - 잠금(mutual exclusion 제공): critical region 에 한 번에 하나의 스레드만 있음
 - Barriers: : 스레드가 이 지점에 도달할 때까지 기다림



Shared address space expression of solver (pseudocode in SPMD execution model)

```
int    n;           //n: 그리드의 크기.
bool   done = false; //done: 계산 종료 여부를 나타내는 플래그.
float  diff = 0.0;   //diff: 각 반복 단계에서 그리드 값의 총 변화량을 저장하는 변수.
LOCK   myLock;       //myLock: 여러 스레드가 diff 변수에 동시에 접근하여 값을 변경할 때 데이터 경쟁(race condition)을 방지하기 위한 잠금 장치 (Lock).
BARRIER myBarrier;  //myBarrier: 모든 스레드가 특정 지점에서 동기화될 때까지 기다리게 하는 장벽 (Barrier).
```

// allocate grid

```
float* A = allocate(n+2, n+2);
```

```
void solve(float* A) {
```

```
    float myDiff;
```

```
    int threadId = getThreadId();
```

```
    int myMin = 1 + (threadId * n / NUM_PROCESSORS);
```

```
    int myMax = myMin + (n / NUM_PROCESSORS)
```

```
    while (!done) {
```

```
        float myDiff = 0.f;
```

```
        diff = 0.f;
```

```
        barrier(myBarrier, NUM_PROCESSORS);
```

```
        for (j=myMin to myMax) {
```

```
            for (i = red cells in this row) {
```

```
                float prev = A[i,j];
```

```
                A[i,j] = 0.2f * (A[i-1,j] + A[i,j-1] + A[i,j] + A[i+1,j], A[i,j+1]);
```

```
                myDiff += abs(A[i,j] - prev));
```

```
            }
```

```
            lock(myLock);
```

```
            diff += myDiff;
```

```
            unlock(myLock);
```

```
            barrier(myBarrier, NUM_PROCESSORS);
```

```
            if (diff/(n*n) < TOLERANCE) // check convergence, all threads get same answer
```

```
                done = true;
```

```
            barrier(myBarrier, NUM_PROCESSORS);
```

```
        }
```

```
    }
```

전역 변수(global variables)라고 가정
(모든 스레드에서 액세스 가능)

solve() 함수가 모든 스레드에서 실행된다고 가정
(SPMD 스타일)

threadId의 값은 각 SPMD 인스턴스마다 다름:
이 값을 사용하여 작업할 그리드 영역을 계산

각 스레드는 업데이트할 책임이 있는 행을 계산

이 lock는 여기서 뭐하는 거죠 ?????

그리고 이러한 barriers?

동기화 (Synchronization)

•**Lock (Mutex):** diff와 같은 공유 자원에 여러 스레드가 동시에 접근하여 값을 변경할 때 발생할 수 있는 문제를 막기 위해 사용. 코드가 lock()과 unlock() 사이의 임계 영역(critical section)에 있을 때는 다른 스레드는 접근할 수 없음.

•**Barrier:** 모든 스레드가 특정 계산 단계를 완료한 후에 다음 단계로 넘어가도록 보장함. 예를 들어, 모든 스레드가 'red' 셀 계산을 끝내야 'black' 셀 계산 (코드에는 생략됨) 또는 수렴 조건 확인을 할 수 있음.

Synchronization in a shared address space

Shared address space model (abstraction)

- 스레드(Threads)는 공유 주소 공간[shared address space](공유 변수)의 위치에서 읽기/쓰기를 통해 통신.
- 스레드가 시작될 때 $x=0$ 이라고 가정.

Thread 1:

```
// Do work here...
```

```
// write to address holding  
// contents of variable x  
x = 1;
```

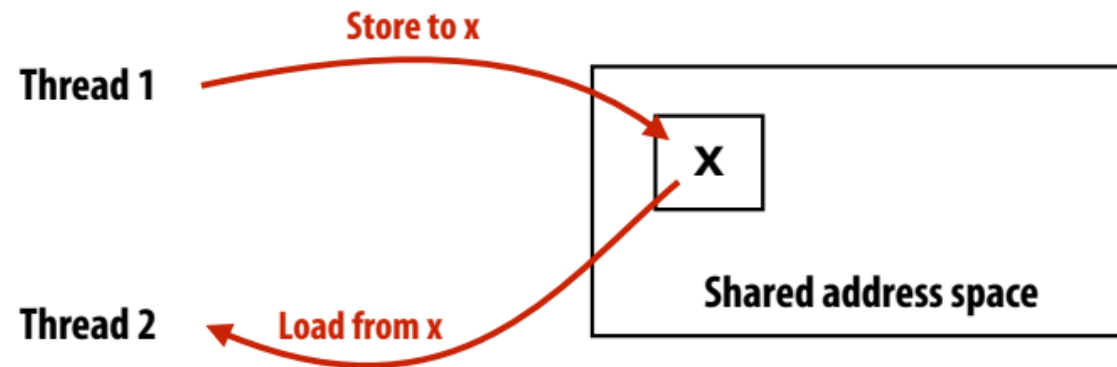
- Thread 1은 어떤 작업을 수행한 후 공유 변수 x 에 1을 저장 ($x = 1;$)

Thread 2:

```
void foo(int* x) {
```

```
// read from addr storing  
// contents of variable x  
while (x == 0) {}  
print x;  
}
```

- Thread 2는 공유 변수 x 의 값이 0이 아닐 때까지 기다리다가 (`while (x == 0) {}`), 값이 바뀌면 (즉, Thread 1이 값을 1로 변경하면) x 의 값을 읽어서 출력 (`print x;`).
- 이 과정에서 Thread 1이 x 에 값을 쓰는 행위(Store)와 Thread 2가 x 의 값을 읽는 행위(Load) 자체가 스레드 간의 통신 역할을 함.



(Communication operations shown in red)

공유 변수에 대한 접근 조정 (Coordinating access to shared variables with synchronization)

Shared (among all threads) variables:

```
int x = 0;  
Lock my_lock;
```

- 병렬 프로그래밍 환경에서는 여러 스레드(또는 프로세스)가 동일한 메모리 공간에 있는 변수(공유 변수)에 동시에 접근하여 읽거나 쓰기 가능.
- 이때 여러 스레드가 순서에 상관없이 공유 변수에 접근하게 되면 예상치 못한 문제가 발생할 수 있는데, 이를 ****경쟁 상태(Race Condition)****라고 함



- 경쟁 상태를 방지하고 공유 변수에 대한 스레드들의 접근을 안전하게 조정하기 위한 **동기화(Synchronization)** 기법

Thread 1:

```
mylock.lock();  
x++;  
mylock.unlock();  
  
print(x);
```

동기화 기법: Lock (뮤텍스): Lock은 공유 자원에 한 번에 하나의 스레드만 접근하도록 허용하는 메커니즘

Thread 2:

```
my_lock.lock();  
x++;  
my_lock.unlock();  
  
print(x);
```

- 왜 동기화가 필요한가?**
- 두 개의 스레드가 x라는 공유 변수의 값을 1증가시키는 연산을 수행한다고 가정해보자.
 - 초기 x의 값이 0이라고 할 때, 각 스레드는 다음과 같은 세 단계를 거쳐 변수를 수정.

- 메모리에서 x의 값을 레지스터로 읽어온다..
- 레지스터에 있는 값에 1을 더한다.
- 레지스터에 있는 값을 메모리의 x에 쓴다.

- 만약 두 스레드가 이 과정을 동시에 수행할 때, 다음과 같은 순서로 실행될 수 있음.
- 스레드 1이 x(값:0)를 읽어와 레지스터에 저장.
 - 스레드 2가 x(값:0)를 읽어와 레지스터에 저장.
 - 스레드 1이 레지스터 값에 1을 더합니다 (값: 1).
 - 스레드 2가 레지스터 값에 1을 더합니다 (값: 1).
 - 스레드 1이 레지스터 값 (값: 1)을 x에 저장.
 - 스레드 2가 레지스터 값 (값: 1)을 x에 저장.



- 결과적으로 x의 값은 1이 됨. 하지만 우리가 기대한 결과는 각 스레드가 한 번씩 1을 더했으므로 최종적으로 x의 값이 2가 되는 것임.
- 이처럼 스레드 실행 순서에 따라 결과가 달라지는 문제가 발생하는데, 이것이 경쟁 상태임

공유 변수에 대한 접근 조정 (Coordinating access to shared variables with synchronization)

Shared (among all threads) variables:

```
int x = 0;  
Lock my_lock;
```

- 병렬 프로그래밍 환경에서는 여러 스레드(또는 프로세스)가 동일한 메모리 공간에 있는 변수(공유 변수)에 동시에 접근하여 읽거나 쓰기 가능.
- 이때 여러 스레드가 순서에 상관없이 공유 변수에 접근하게 되면 예상치 못한 문제가 발생할 수 있는데, 이를 ****경쟁 상태(Race Condition)****라고 함



- 경쟁 상태를 방지하고 공유 변수에 대한 스레드들의 접근을 안전하게 조정하기 위한 **동기화(Synchronization)** 기법

Thread 1:

```
mylock.lock();  
x++;  
mylock.unlock();  
  
print(x);
```

동기화 기법: Lock (뮤텍스): Lock은 공유 자원에 한 번에 하나의 스레드만 접근하도록 허용하는 메커니즘

Thread 2:

```
my_lock.lock();  
x++;  
my_lock.unlock();  
  
print(x);
```

왜 동기화가 필요한가?

- 두 개의 스레드가 x라는 공유 변수의 값을 1증가시키는 연산을 수행한다고 가정해보자.
- 초기 x의 값이 0이라고 할 때, 각 스레드는 다음과 같은 세 단계를 거쳐 변수를 수정.

- 메모리에서 x의 값을 레지스터로 읽어온다..
- 레지스터에 있는 값에 1을 더한다.
- 레지스터에 있는 값을 메모리의 x에 쓴다.

만약 두 스레드가 이 과정을 동시에 수행할 때, 다음과 같은 순서로 실행될 수 있음.

- 스레드 1이 x(값:0)를 읽어와 레지스터에 저장.
- 스레드 2가 x(값:0)를 읽어와 레지스터에 저장.
- 스레드 1이 레지스터 값에 1을 더합니다 (값: 1).
- 스레드 2가 레지스터 값에 1을 더합니다 (값: 1).
- 스레드 1이 레지스터 값 (값: 1)을 x에 저장.
- 스레드 2가 레지스터 값 (값: 1)을 x에 저장.



- 결과적으로 x의 값은 1이 됨. 하지만 우리가 기대한 결과는 각 스레드가 한 번씩 1을 더했으므로 최종적으로 x의 값이 2가 되는 것임.
- 이처럼 스레드 실행 순서에 따라 결과가 달라지는 문제가 발생하는데, 이것이 경쟁 상태임

공유 변수에 대한 접근 조정 (Coordinating access to shared variables with synchronization)

```
mylock.lock(); // Lock 획득
```

```
    x++;    // 공유 변수 x 접근 (임계 영역)
```

```
mylock.unlock(); // Lock 해제
```

- 이 코드를 사용하면 다음과 같이 실행.

- * 스레드 1이 `mylock`을 획득

- * 스레드 1이 `x`를 읽고, 1을 더하고, `x`에 씁니다. (x는 이제 1)

- * 스레드 1이 `mylock`을 해제

- * 스레드 2가 `mylock`을 획득 (스레드 1이 해제했으므로).

- * 스레드 2가 `x`를 읽고 (값: 1), 1을 더하고, `x`에 씁니다. (x는 이제 2)

- * 스레드 2가 `mylock`을 해제

- 이렇게 Lock을 사용하면 공유 변수에 대한 접근을 원자적(Atomic)으로 만들 수 있음. 원자적 연산이란 어떤 작업이 더 이상 나눌 수 없는 하나의 단위로 실행되어 도중에 다른 스레드에 의해 방해받지 않음을 의미.
- `lock()`과 `unlock()` 함수를 사용하여 Lock을 제어하는 예시를 보여줌. 또한, 하드웨어적으로 지원되는 원자적 연산이나 일부 언어에서 제공하는 `atomic` 블록 등을 통해 원자성을 보장.
- 요약하자면, 병렬 환경에서 여러 스레드가 공유 변수에 동시에 접근할 때는 경쟁 상태가 발생할 수 있으며, 이를 해결하기 위해 Lock과 같은 동기화 기법을 사용하여 공유 변수에 대한 접근을 조정함. 이를 통해 프로그램의 정확성을 보장할 수 있음.

Review: why do we need mutual exclusion?

- 각 스레드가 실행:
 - 메모리 내 위치에서 변수 x 의 값을 레지스터 $r1$ 로 로드.
(레지스터의 메모리에 값의 복사본을 저장).
 - 레지스터 $r2$ 의 내용을 레지스터 $r1$ 에 덧셈.
 - 레지스터 $r1$ 의 값을 프로그램 변수 x 를 저장하는 주소에 저장.
- 한 가지 가능한 인터리빙: (시작 값을 $x=0, r2=1$ 로 함)

어떤 문제가 생기는 걸까요???

T1	T2
$r1 \leftarrow x$	T1 reads value 0
	T2 reads value 0
$r1 \leftarrow r1 + r2$	T1 sets value of its $r1$ to 1
	T2 sets value of its $r1$ to 1
$x \leftarrow r1$	T1 stores 1 to address of x
	T2 stores 1 to address of x

- 이 세 개의 명령어 집합은 “atomic”여야 함.

원자성(atomicity)을 보존하는 메커니즘 예시

- 중요 섹션 주변의 뮤텝스 잠금/잠금 해제

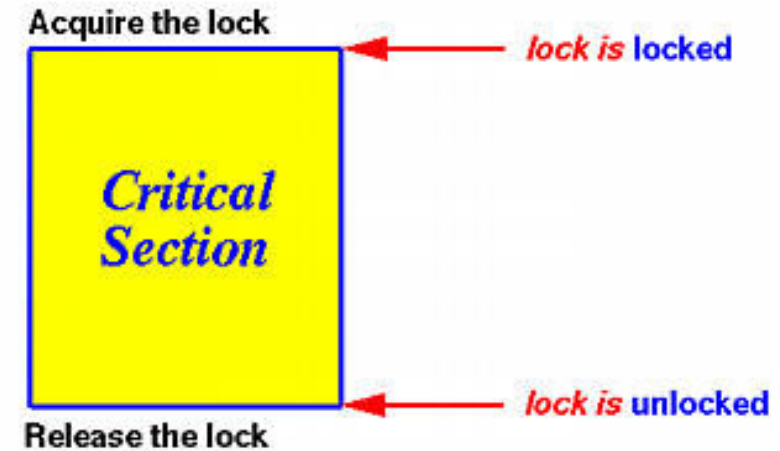
```
mylock.lock();  
// critical section  
mylock.unlock();
```

- 일부 언어는 코드 블록의 원자성을 최고 수준으로 지원.

```
atomic {  
    // critical section  
}
```

- 하드웨어 지원 원자 읽기-수정-쓰기 연산을 위한 내재성

```
atomicAdd(x, 10);
```



Summary: shared address space model

- 스레드는 다음과 같이 통신:
 - 공유 주소 공간(shared address space)에서 공유 변수(shared variables)에 대한 읽기/쓰기
 - 스레드 간 통신(Communication between threads)은 메모리 로드/스토어에 내재되어 있음.
 - 동기화 프리미티브 조작
 - 예: 잠금 사용을 통한 상호 배제(mutual exclusion) 보장
- 이는 순차적 프로그래밍의 자연스러운 확장입니다.
 - 사실, 지금까지 수업에서 논의한 모든 내용은 공유 주소 공간을 가정!

```
int    n;           // grid size
bool   done = false;
float  diff = 0.0;
LOCK   myLock;
BARRIER myBarrier;

// allocate grid
float* A = allocate(n+2, n+2);

void solve(float* A) {
    float myDiff;
    int threadId = getThreadId();
    int myMin = 1 + (threadId * n / NUM_PROCESSORS);
    int myMax = myMin + (n / NUM_PROCESSORS)

    while (!done) {
        float myDiff = 0.f;
        diff = 0.f;
        barrier(myBarrier, NUM_PROCESSORS);
        for (j=myMin to myMax) {
            for (i = red cells in this row) {
                float prev = A[i,j];
                A[i,j] = 0.2f * (A[i-1,j] + A[i,j-1] + A[i,j] + A[i+1,j], A[i,j+1]);
                LOCK(myLock);
                diff += abs(A[i,j] - prev);
                UNLOCK(myLock);
            }
        }
        barrier(myBarrier, NUM_PROCESSORS);
        if (diff/(n*n) < TOLERANCE)
            done = true;
        barrier(myBarrier, NUM_PROCESSORS);
    }
}
```

threadId의 값은 각 SPMD 인스턴스마다 다름:
이 값을 사용하여 작업할 그리드 영역을 계산

각 스레드는 업데이트할 책임이 있는 행을 계산

Hint

이 구현에서 잠재적인 성능 문제를 발견
하셨나요?

```
int    n;           // grid size
bool   done = false;
float  diff = 0.0;
LOCK   myLock;
BARRIER myBarrier;

// allocate grid
float* A = allocate(n+2, n+2);

void solve(float* A) {
    float myDiff;
    int threadId = getThreadId();
    int myMin = 1 + (threadId * n / NUM_PROCESSORS);
    int myMax = myMin + (n / NUM_PROCESSORS)

    while (!done) {
        float myDiff = 0.f;
        diff = 0.f;
        barrier(myBarrier, NUM_PROCESSORS);
        for (j=myMin to myMax) {
            for (i = red cells in this row) {
                float prev = A[i,j];
                A[i,j] = 0.2f * (A[i-1,j] + A[i,j-1] + A[i,j] + A[i+1,j], A[i,j+1]);
                myDiff += abs(A[i,j] - prev);
            }
            lock(myLock);
            diff += myDiff;
            unlock(myLock);
        }
        barrier(myBarrier, NUM_PROCESSORS);
        if (diff/(n*n) < TOLERANCE)           // check convergence, all threads get same answer
            done = true;
        barrier(myBarrier, NUM_PROCESSORS);
    }
}
```

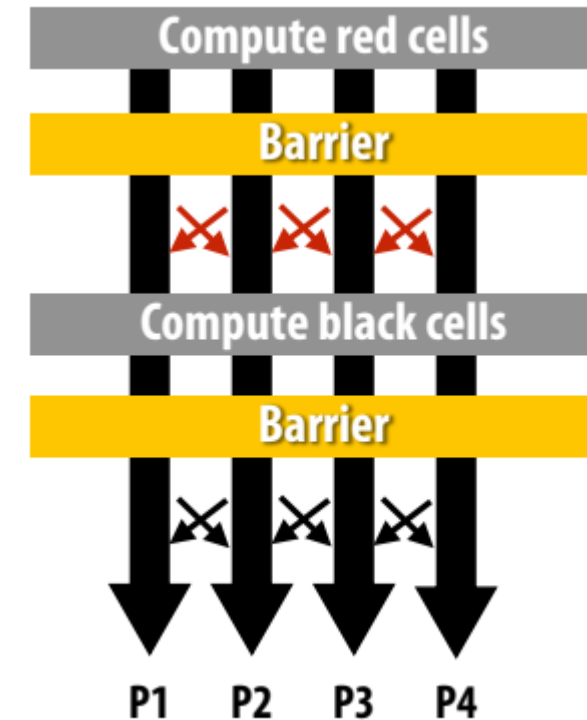
로컬에서 부분 합으로 누적하여 성능을 개선
한 다음 반복이 끝날 때 global reduction를 완료
함

worker당 부분 합계 계산

이제 (i,j) 루프 반복당 한 번이 아니라 스레드당 한 번만
잠금

Barrier synchronization primitive

- `barrier(num_threads)`
- 배리어(Barriers)는 의존성(dependencies)을 표현하는 보수적인 방법.
- 배리어(Barriers)는 연산을 여러 단계로 나눔.
- 배리어(Barriers) 이전의 모든 스레드에 의한 모든 계산은 배리어(Barriers)가 시작되기 전에 배리어(Barriers) 이후의 모든 스레드에서 계산이 완료되어야 함.
 - 즉, 배리어(Barriers) 이후의 모든 계산은 배리어(Barriers) 이전의 모든 계산에 의존하는 것으로 가정.



Shared address space solver

Why are there three barriers?

```
int    n;           // grid size
bool   done = false;
float  diff = 0.0;
LOCK   myLock;
BARRIER myBarrier;

// allocate grid
float* A = allocate(n+2, n+2);

void solve(float* A) {
    float myDiff;
    int threadId = getThreadId();
    int myMin = 1 + (threadId * n / NUM_PROCESSORS);
    int myMax = myMin + (n / NUM_PROCESSORS)

    while (!done) {
        float myDiff = 0.f;
        diff = 0.f;
        barrier(myBarrier, NUM_PROCESSORS);
        for (j=myMin to myMax) {
            for (i = red cells in this row) {
                float prev = A[i,j];
                A[i,j] = 0.2f * (A[i-1,j] + A[i,j-1] + A[i,j] + A[i+1,j], A[i,j+1]);
                myDiff += abs(A[i,j] - prev);
            }
            lock(myLock);
            diff += myDiff;
            unlock(myLock);
            barrier(myBarrier, NUM_PROCESSORS);
            if (diff/(n*n) < TOLERANCE) // check convergence, all threads get same answer
                done = true;
            barrier(myBarrier, NUM_PROCESSORS);
        }
    }
}
```

Shared address space solver: one barrier

```
int    n;           // grid size
bool   done = false;
LOCK   myLock;
BARRIER myBarrier;
float  diff[3];     // global diff, but now 3 copies

float *A = allocate(n+2, n+2);

void solve(float* A) {
    float myDiff;    // thread local variable
    int  index = 0;  // thread local variable

    diff[0] = 0.0f;
    barrier(myBarrier, NUM_PROCESSORS); // one-time only: just for init

    while (!done) {
        myDiff = 0.0f;
        //
        // perform computation (accumulate locally into myDiff)
        //
        lock(myLock);
        diff[index] += myDiff;    // atomically update global diff
        unlock(myLock);
        diff[(index+1) % 3] = 0.0f;
        barrier(myBarrier, NUM_PROCESSORS);
        if (diff[index]/(n*n) < TOLERANCE)
            break;
        index = (index + 1) % 3;
    }
}
```

아이디어:

연속적인 루프 반복에서 서로 다른 diff 변수를 사용하여 종속성 제거하기

종속성 제거를 위해 footprint의 Trade off
(일반적인 병렬 프로그래밍 기법)

Grid solver implementation in two programming models

- Data-parallel programming model

- 동기화 (Synchronization):

- 주로 시스템이 관리.
 - forall 같은 구문은 논리적으로는 하나의 제어 흐름처럼 보이지만, 그 안의 반복 작업들이 시스템에 의해 병렬로 실행.
 - forall 루프의 끝에는 암묵적인 장벽(implicit barrier)이 있어서, 루프 내의 모든 병렬 작업이 완료될 때까지 기다린 후 다음 코드로 넘어감. 프로그래머가 barrier 같은 명령을 직접 쓸 필요가 줄어듦.

- 통신 (Communication)

- - 공유 주소 공간 모델처럼 메모리 로드/스토어를 통한 암묵적 통신이 기본
 - - reduce (모든 값을 더하는 등)와 같이 여러 데이터 조각에 걸친 복잡한 통신 패턴을 위한 특별한 내장 함수(built-in primitives)를 제공하여 시스템이 효율적으로 처리하도록 함.

- 공유 주소 공간 모델 (Shared Address Space)

- 동기화 (Synchronization):

- 프로그래머가 명시적으로 관리, 공유 변수(예: diff)에 여러 스레드가 동시에 접근하여 값을 변경할 때 데이터가 깨지는 것을 막기 위해 락(locks)을 사용하여 상호 배제(mutual exclusion)를 보장해야 함
 - 계산의 특정 단계(phase)들이 순서대로 진행되도록 (예: 모든 Red 셀 업데이트 후 Black 셀 업데이트 시작) 스레드들을 기다리게 하는 배리어(barriers)를 사용하여 의존성을 표현

- 통신 (Communication)

- 공유 변수에 값을 쓰고 읽는 메모리 로드/스토어를 통해 암묵적으로 이루어짐

Grid solver implementation in two programming models

특징	데이터 병렬 모델 (Data-Parallel)	공유 주소 공간 모델 (Shared Address Space)
주요 초점	데이터에 대한 병렬 연산 정의	스레드 간 작업 분담 및 협력 방식 정의
동기화 관리	주로 시스템 (암묵적) (forall 끝 등)	프로그래머 (명시적) (lock, barrier 등)
통신	암묵적(로드/스토어) + 내장 함수 (reduce 등)	암묵적 (로드/스토어)
프로그래머 부담	병렬화 세부 구현 부담 적음	동기화 등 세부 구현 및 관리 부담 큼
제어 수준	상대적으로 낮음	상대적으로 높음

- **암달의 법칙**

- 병렬 처리로 인한 전체 최대 속도 향상은 프로그램의 serial execution 에 따라 제한됨

- **병렬 프로그램 생성의 측면**

- 독립적인 작업을 생성하기 위한 분해, 작업자에게 작업 할당, 오케스트레이션(작업자별 작업 처리 조정), 하드웨어에 매핑하기

- **오늘 집중하기: 종속성 파악하기(identifying dependencies)**

- **곧 집중할 내용: identifying locality, reducing synchronization**